CHAINLENS: EXPLAINABLE ETHEREUM WALLET RISK SCORING

A bounded-window, rule-based pipeline for auditable crypto-risk signals

A COURSE PROJECT REPORT

submitted by

Christo Philip Mathew (MUT24CA031)
James S Alapatt (MUT24CS081)
Nevin Alex Koshy (MUT24CS115)
Richard Shaji Mekkaden (MUT24CS127)

as part of the course

PECST525 – DATA MINING

to

The APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree

of

Bachelor of Technology
in
CSE, CSE (AI)



Department of Artificial Intelligence & Data Science
Muthoot Institute of Technology and Science
Kochi – 682308

OCTOBER 2026

CERTIFICATE

This is to certify that the report entitled **"ChainLens: Explainable Ethereum Wallet Risk Scoring"**, *submitted by* **Christo Philip Mathew (MUT24CA031), James S Alapatt (MUT24CS081), Nevin Alex Koshy (MUT24CS115), and Richard Shaji Mekkaden (MUT24CS127)**, *to Muthoot Institute of Technology and Science, Kochi for the award of the degree of Bachelor of Technology in CSE, CSE (AI), is a bonafide record of the course project work carried out by them, as part of the course PECST525 Data Mining, under my supervision and guidance.*

Mrs. Parvathy E N
Assistant Professor
Department of Artificial Intelligence & Data Science
Faculty-in-charge

Place: Kochi
Date: 29/10/2026

DECLARATION

We hereby declare that the report entitled **"ChainLens: Explainable Ethereum Wallet Risk Scoring"**, submitted to Muthoot Institute of Technology and Science, Kochi, is a record of the original work carried out by us as part of the course PECST525 Data Mining. The analyses, implementation, and interpretations presented in this report are our own. All external data sources, software libraries, algorithms, and scholarly works that influenced this work have been acknowledged in the text and listed in the references. This report has not been submitted, either in part or in full, for the award of any other degree or diploma at this or any other institution.

Place: Kochi

Date: 29/10/2026

Christo Philip Mathew (MUT24CA031)

James S Alapatt (MUT24CS081)

Nevin Alex Koshy (MUT24CS115)

Richard Shaji Mekkaden (MUT24CS127)

ACKNOWLEDGEMENT

We express our sincere gratitude to Mrs. Parvathy E N, Assistant Professor, Department of Artificial Intelligence & Data Science, Muthoot Institute of Technology and Science, Kochi, for her guidance throughout this course project. Her feedback on problem formulation, the evaluation design, and the presentation of results helped us keep the work rigorous and readable.

We are grateful to the Head of the Department and the faculty members of the Department of Artificial Intelligence & Data Science for providing the academic environment and computing facilities that made this work possible. We also thank our classmates and reviewers who examined early demonstrations of the dashboard and pointed out inconsistencies in the evidence views.

Finally, we thank our families for their encouragement and support during the project. Any shortcomings that remain in the design, implementation, or interpretation of the results are ours alone.

The Authors

October 2026

ABSTRACT

Ethereum's public ledger supports accountability, but its scale and pseudonymity make it difficult for analysts to separate ordinary high-volume activity from conduct that warrants investigation. Existing commercial risk scores are often opaque: a wallet receives a number without a defensible account of the evidence behind it. This report presents ChainLens, an explainable wallet risk-scoring pipeline that combines graph heuristics, association-rule mining, density clustering, and community detection with a locked additive scoring contract. Every score is accompanied by a per-wallet trace that records the rules that fired, their evidence strength, their point contribution, and any benign-entity downweighting.

The system was evaluated on an artifact-backed real-mode run of 28 October 2026. The run ingested 379,023 raw records, retained 368,113 clean transactions, engineered 14 behavioural features for 81,756 wallets, and built a directed graph with 99,706 edges. Five rules were applied: circular flow (R1), scatter–gather smurfing (R2), one-hop sanctioned exposure (R3), continuous value-weighted PageRank (R4), and Apriori profile matching (R5). R1 identified 97 length-two cycles involving 163 wallets, R3 flagged 2,014 wallets with a mean evidence strength of 0.963, and R4 flagged 7,758 wallets in the continuous upper tail. R5 produced no rules because the best candidate lift of 1.394 fell below the locked threshold of 2.0. The resulting distribution was one High-tier wallet, 2,094 Medium-tier wallets, and 79,661 Low-tier wallets.

On the held-out case-control test sample of 538 positives and 599 negatives, the High threshold (score ≥ 60) achieved precision, recall, and F1 of 0, 0, and 0; the Medium threshold (score ≥ 25) achieved 54.55% precision, 2.23% recall, and 4.29% F1, with an average precision of 0.6181. These are sample metrics, not population estimates. The results show that the pipeline produces a usable, auditable triage queue while also exposing the cost of a high evidence threshold. By making illicit-finance signals inspectable, the work contributes primarily to Sustainable Development Goal 16 (peace, justice, and strong institutions), with secondary relevance to Goals 9 and 4.

Keywords: blockchain forensics, Ethereum, anti-money laundering, explainable risk scoring, graph analytics, association rule mining, PageRank, auditability, SDG 16.

SDG MAPPING

The 2030 Agenda treats peaceful institutions and safe digital finance as interdependent goals [1]. ChainLens is mapped primarily to **Goal 16** because it makes illicit financial flows and sanctions exposure visible and contestable. Goals 9 and 4 are secondary mappings. Table 1 states the targets and the project's contribution.

Table 1: Mapping of ChainLens to the Sustainable Development Goals.

SDG	Targets	Justification
Goal 16: Peace, justice, and strong institutions *(primary)*	16.4, 16.5, 16.6, 16.10	Detects circular flows, smurf-like dispersion, and direct exposure to frozen sanctions seeds—signals of illicit financial flows and corruption. Locked weights, a hashed rule contract, and per-wallet evidence traces make every escalation reviewable, supporting transparent and accountable institutions.
Goal 9: Industry, innovation, and infrastructure	9.1, 9.5	Treats the transaction graph as critical digital infrastructure that can be monitored without opaque models. The reproducible graph-analytics pipeline is a small contribution to trustworthy fintech infrastructure.
Goal 4: Quality education	4.4, 4.7	The implementation is documented and reproducible on a fixed public artifact set, and the dashboard turns abstract rules into inspectable evidence for teaching blockchain forensics, graph mining, and responsible algorithm design.

Goal 16 is primary because success is not predictive accuracy alone. A score is useful only when an analyst can reconstruct why a wallet was escalated, separate a sanctions link from benign high-volume activity, and challenge the result. Reporting false positives, the case-control caveat, and the degenerate High-tier result is therefore part of the accountability contribution.

CONTENTS

DECLARATION i

ACKNOWLEDGEMENT ii

ABSTRACT iii

SDG MAPPING iv

LIST OF FIGURES vii

LIST OF TABLES viii

1 INTRODUCTION 1

1.1 BACKGROUND AND MOTIVATION 1

1.2 PROBLEM STATEMENT 2

1.3 OBJECTIVES 3

1.4 SCOPE AND DELIMITATIONS 3

1.5 ORGANISATION OF THE REPORT 4

2 LITERATURE REVIEW 5

2.1 EXISTING SYSTEMS 5

2.2 REVIEW OF RELATED WORKS 6

2.3 RESEARCH GAPS 10

2.4 SUMMARY 11

3 DATASET AND PREPROCESSING 12

3.1 DATASET DESCRIPTION 12

3.2 DATA SOURCES AND LABELLING 13

3.3 DATA CLEANING AND DEDUPLICATION 14

3.4 FEATURE ENGINEERING 15

3.5 LABELLED COHORT AND TRAIN/TEST SPLIT 16

3.6 FEATURE SCALING AND LEAKAGE CONTROL 17

3.7 SUMMARY 18

4 PROPOSED SYSTEM AND METHODOLOGY 19

4.1 SYSTEM OVERVIEW AND ARCHITECTURE 19

4.2 THEORETICAL BACKGROUND 20

4.3 DATA MINING ALGORITHMS 22

4.4 COMPOSITE SCORING AND RISK TIERING 27

4.5 SYSTEM WORKFLOW 28

4.6 EVALUATION METHODOLOGY 29

4.7 SUMMARY 30

5 IMPLEMENTATION AND RESULTS 31

5.1 IMPLEMENTATION ENVIRONMENT AND TECHNOLOGY STACK . . 31

5.2 MODULE IMPLEMENTATION 32

5.3 DASHBOARD IMPLEMENTATION 34

5.4 RUN SUMMARY 35

5.5 PATTERN DETECTION RESULTS 35

5.6 RISK SCORING RESULTS 37

5.7 DASHBOARD RESULTS 39

5.8 PERFORMANCE EVALUATION 44

5.9 ABLATION STUDY 46

5.10 DISCUSSION 47

5.11 PROVENANCE AND EXPORT 48

5.12 SUMMARY 49

6 CONCLUSION AND FUTURE SCOPE 50

6.1 CONCLUSION 50

6.2 CONTRIBUTIONS 51

6.3 LIMITATIONS 51

6.4 FUTURE SCOPE 52

6.5 SUSTAINABLE DEVELOPMENT GOAL 16 53

6.6 CONCLUDING REMARKS 54

LIST OF FIGURES

4.1 Decoupled architecture of ChainLens. 19

4.2 End-to-end workflow of the reported run. 28

5.1 Priority triage queue. 40

5.2 Risk and signal distributions. 40

5.3 Triage filters and result table. 41

5.4 Wallet dossier with rule evidence. 42

5.5 Transaction evidence for the top wallet. 42

5.6 R1 pattern output. 43

5.7 Transaction subgraph and community structure. 44

5.8 Evaluation operating points. 45

5.9 Leave-one-rule-out ablation. 47

5.10 Artifact inventory and provenance. 49

LIST OF TABLES

1 Mapping of ChainLens to the Sustainable Development Goals. iv

2.1 Representative works and the design gap addressed by ChainLens. 10

3.1 Scale and composition of the reported real-mode run. 13

3.2 The fourteen behavioural features engineered for each wallet. 15

3.3 Label composition and leakage-safe address split. 17

4.1 Locked rule weights and principal parameters. 27

4.2 Risk tiers and their interpretation. 28

5.1 Implementation stack and rationale. 31

5.2 Provenance of the reported run (28 October 2026). 35

5.3 Pattern detection output for the reported run. 36

5.4 Final tier distribution and score concentration. 37

5.5 Five highest-scoring wallets and their rule contributions. 38

5.6 Number of scoring rules hit per wallet. 39

5.7 Operating-point metrics on the held-out case-control test sample. 44

5.8 Confusion matrices at the two operating points. 45

5.9 Leave-one-rule-out ablation at the High threshold. 46

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Public blockchains are, in principle, the most auditable financial records ever built. Every transfer is time-stamped, permanently stored, and independently verifiable. In practice, this transparency creates a different problem: the volume and pseudonymity of the ledger make it difficult to decide which addresses deserve attention. A compliance analyst who receives a list of tens of thousands of wallets cannot inspect each one manually, and a risk score that cannot be explained is difficult to defend to a supervisor, an auditor, or a regulated counterparty.

The problem is not merely technical. Anti-money-laundering (AML) and counter-financing-of-terrorism (CFT) obligations require institutions to identify beneficial owners, monitor transactions, and report suspicious activity. The Financial Action Task Force (FATF) recommends a risk-based approach in which higher-risk relationships receive deeper scrutiny [2]. For virtual assets, this means that an institution must be able to justify why a wallet was escalated, not only that a model assigned it a high number. Sanctions compliance adds a second layer: direct exposure to a designated address is a legally material fact, and the evidence must be traceable to specific transfers rather than to an opaque embedding [3].

Ethereum is a useful setting for this problem because its account model produces an explicit directed graph of value transfers. Graph structure carries information that tabular features miss: cycles can indicate layering, a scatter of small transfers can indicate structuring, and a dense community can reveal a cluster of addresses that behave together. At the same time, graph structure is full of benign high-volume hubs. Exchanges, bridges, routers, and stablecoin contracts appear in an enormous number of transfers and can dominate any centrality measure. A naive detector therefore produces many false positives precisely where the data is richest.

ChainLens was built around the belief that explainability is an engineering requirement, not a presentation layer added after modelling. The system uses five transparent rules, each

grounded in a published typology or a documented data-mining algorithm, and combines them with a locked additive scoring contract. Every score is accompanied by a trace containing the evidence that produced it: the cycle path for a circular flow, the counterparties and temporal ordering for a smurf motif, the seed address for a sanctions link, the percentile for a PageRank contribution, or the antecedent for an association rule. The dashboard is deliberately read-only so that the analyst sees the same artifacts that were evaluated.

The project also has a development goal. Blockchain forensics is often taught through either high-level compliance theory or low-level cryptographic primitives, with little attention to the messy middle where data engineering, graph algorithms, evaluation design, and interface design meet. A reproducible pipeline that runs on a fixed public artifact set and exposes its evidence is a useful teaching instrument for that middle layer. This is why the work is mapped primarily to Sustainable Development Goal 16, with secondary relevance to Goals 9 and 4.

1.2 PROBLEM STATEMENT

Ethereum's public transaction data is valuable for compliance and security analysis, but its scale makes manual review impossible. Existing tools often produce opaque, black-box risk scores with no auditable evidence, leaving analysts unable to justify a decision or to distinguish a genuinely suspicious pattern from a benign high-volume entity. The problem is compounded by class imbalance and label scarcity: confirmed illicit addresses are rare, labels are incomplete, and a model that appears accurate on a balanced benchmark may be unusable in an operational queue.

The specific problem addressed by this project is therefore threefold. First, the system must convert raw Etherscan records into a consistent behavioural representation without losing the fixed-point precision required for financial amounts. Second, it must detect a small set of interpretable risk patterns while suppressing obvious benign hubs. Third, it must evaluate the resulting scores honestly, including the possibility that a high threshold produces no true positives on the available test sample.

A fourth, more practical requirement follows from the academic setting: the complete pipeline must be reproducible from stored artifacts and documented configuration. A score that cannot be regenerated is not auditable, and an evaluation that depends on a hidden

notebook state is not a result. The reported run is therefore anchored to a timestamped artifact directory containing the rule hash, scaler version, package versions, and counts used in this report.

1.3 OBJECTIVES

The project pursued the following objectives:

1. To collect and analyse Ethereum transaction data within a bounded time window using publicly available sources.
2. To design and implement an explainable, rule-based risk-scoring system (R1–R5) for identifying high-risk wallets.
3. To generate an auditable score trace for each wallet with transparent evidence and no black-box model in the decision path.
4. To evaluate the effectiveness of the scoring system on a leakage-safe train/test split using precision, recall, F1, average precision, and a leave-one-rule-out ablation.
5. To develop a user-friendly, read-only web application that displays wallet risk scores, transaction insights, and supporting evidence for better decision making.

1.4 SCOPE AND DELIMITATIONS

The scope of the reported implementation is a batch analysis of a fixed cohort of 81,756 Ethereum wallets. Transaction ingestion uses the Etherscan V2 API with a per-address cache and a bounded recent window of at most 500 transactions per address. The feature layer computes 14 behavioural measures per wallet. Detection combines five rules with DBSCAN density clustering and Louvain community detection; the latter two provide investigative signals, while only R1–R5 contribute points. Scoring uses locked weights of 25, 20, 30, 15, and 20, a known-entity downweight factor of 0.3, and a final clamp to the range 0–100. Wallets are assigned to High (≥ 60), Medium (25–59), or Low (0–24) tiers.

Several delimitations are important. The system is not a real-time monitor; it operates on a stored snapshot. It does not establish legal liability or ownership, and a High-tier score is a

prioritisation signal rather than proof of wrongdoing. The labelled cohort is a case-control sample rather than a representative population sample, so the reported precision and recall are sample metrics. The rules are deliberately conservative, which limits recall. R5 produced no association rules on this run, so the five-rule design operated as a four-rule system in practice. These limitations are treated as findings in Chapter 5, not as details to be hidden.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews the systems and research that inform the design, including wash-trading detection, smurfing, sanctions proximity, graph centrality, association-rule mining, density clustering, and community detection. Chapter 3 describes the data sources, cleaning operations, feature engineering, labelling, and split design. Chapter 4 presents the proposed system, the scoring model, the detection algorithms, and the evaluation methodology. Chapter 5 reports the implementation, the artifact-backed results, the dashboard walkthrough, the operating-point metrics, and the ablation study. Chapter 6 concludes with limitations and future scope, including the project's SDG 16 contribution.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEMS

Operational blockchain analytics is dominated by proprietary compliance platforms that cluster addresses, attach risk labels, and expose a score or category to customers. These systems are valuable because they aggregate several data sources and maintain curated sanction and fraud lists, but their scoring logic is usually not available for inspection. An analyst can see that a wallet is "high risk" without seeing which transfer, counterparty, or behavioural pattern produced the label. This is the gap that motivates an explainable, rule-based alternative.

A second class of existing systems is academic. The Elliptic dataset and similar benchmarks provide labelled Bitcoin transaction graphs for supervised learning [4]. Graph neural networks and ensemble classifiers can achieve strong benchmark performance, but they inherit the interpretability problem: a prediction is distributed across learned weights rather than attached to a specific typology. The same tension appears in Ethereum-specific work, where graph features improve detection but also require careful handling of high-degree hubs [5, 6].

A third class is open-source infrastructure. Etherscan provides the transaction API used in this project [7]; NetworkX provides graph construction, strongly connected components, cycle search, and PageRank [8]; DuckDB provides exact decimal aggregation over Parquet [9]; and scikit-learn and MLxtend provide DBSCAN and Apriori [10, 11]. These tools make a transparent pipeline feasible, but they do not by themselves answer the compliance question of how to combine signals into a defensible score.

ChainLens therefore sits between the proprietary and academic extremes. It uses the same broad ingredients as commercial systems—transaction ingestion, behavioural features, graph patterns, labels, and a risk tier—but keeps the decision contract small enough to audit. The five rules are not a complete AML programme. They are a deliberately bounded vocabulary of suspicious patterns, each with a published rationale and a concrete evidence object.

2.2 REVIEW OF RELATED WORKS

2.2.1 Wash Trading and Circular Flow

Victor and Weintraud study wash trading on decentralised exchanges and formalise it as trading with oneself through multiple accounts [12]. Their method first identifies strongly connected components (SCCs) in the directed trading graph and then searches for short cycles within those components. Two-account self-trade pairs are particularly important: the same entity controls both sides of a trade, and the transfer amounts match apart from fees. The paper also argues that a binary flag is less informative than a volume-based measure of how much trading is wash trading.

The ChainLens R1 rule follows the structural part of this work. Tarjan's SCC algorithm reduces the search space, and bounded cycle enumeration looks for lengths between two and five. The value tolerance is tightened to $\pm 2\%$ so that transfers that differ only by gas-fee noise are treated as matching, and the entire cycle must complete within 24 hours. The rule does not attempt to estimate a wash-trading volume ratio; it flags membership and records the cycle path, value ratio, duration, and SCC size. This is a narrower contribution than the original paper, but it is sufficient for a triage signal and produces evidence that an analyst can inspect.

Hu et al. characterise money laundering on Bitcoin as a layered process and use graph-structural features, including loops and PageRank, to identify intermediaries [13]. Their work supports the general proposition that topology matters beyond transaction-level features. It also warns that cyclic structure is not unique to laundering: exchanges, arbitrage, and routing contracts can create loops. ChainLens addresses this partly through the known-entity downweight and partly by reporting cycle evidence rather than treating every loop as equivalent.

2.2.2 Smurfing and the Scatter–Gather Motif

Starnini et al. define smurfing as a temporal scatter–gather motif rather than a simple fan-out [14]. A source disperses funds to several intermediaries, and those intermediaries forward funds to one or more collectors. The ordering matters: the gather edge must occur after the scatter edge, within a bounded window. The paper implements the motif with temporal

database joins, which is substantially faster than subgraph isomorphism on large transaction networks.

This distinction is important because fan-out alone is common and often benign. A payroll wallet, an exchange hot wallet, or an airdrop distributor may send to hundreds of addresses without any intent to obscure the origin of funds. ChainLens R2 therefore uses a two-part gate. The scatter gate requires at least 20 unique counterparties, at least 20 outbound transactions, and a fan-out ratio of at least 0.5. The gather gate then requires that at least two intermediaries, or 10% of the scattering counterparties, forward funds onward after the scatter edge, within 72 hours, to at least one collector. A wallet that only scatters is a candidate, not a confirmed smurf.

The implementation is still a simplification. Starnini et al. emphasise relative velocity and adapt thresholds to the transaction rate of the network, whereas ChainLens uses absolute counts calibrated for the cohort. The 72-hour window is a compromise between the original motif and the 24-hour window used for cycles. These choices are documented in the configuration and exposed in the trace, so a reviewer can see exactly which gate failed.

2.2.3 Sanctions, Proximity, and Dusting

Sanctions compliance is different from pattern detection because the relevant fact is a relationship to a designated entity, not a behavioural similarity. Liu et al. analyse OFAC sanctions against Tornado Cash across 6.79 million Ethereum blocks and report a substantial reduction in sanctioned volume, while noting that sanctioned infrastructure continued to appear in a large share of security incidents [15]. They argue for an impurity-based quantitative score rather than a binary blacklist and warn that dusting attacks can create misleading one-hop links.

ChainLens R3 incorporates three lessons from this literature. First, seeds are drawn from a frozen OFAC snapshot unioned with labelled sanctioned addresses, independent of the train/test split, so a test wallet is not protected by the accident of split membership. Second, transfers below 0.01 ETH are ignored to resist dusting. Third, the rule computes inflow and outflow exposure fractions and uses $\max(\text{exposure}, 0.10)$ as the evidence strength, so a confirmed direct link retains at least 10% of the 30-point weight even when the sanctioned amount is small relative to the wallet's total volume.

The internal literature review also considered two graph-proximity studies that propose path-distance metrics for blockchain fraud prevention. They are not cited here because their author, venue, and publication metadata could not be verified from the available sources. This omission is deliberate: a report that is meant to be auditable should not include references that a reader cannot check. The one-hop design used here is conservative and is supported by the stronger of the verified sanctions studies.

2.2.4 Graph Centrality and AML

PageRank was originally proposed as a ranking method for web pages [16]. In a transaction graph, a value-weighted variant measures how much of the network's flow reaches a node through influential predecessors. Lin et al. apply complex-network analysis to Ethereum transaction records and show that PageRank scores are heavy-tailed and that high-ranking nodes include exchanges, DeFi routers, and bridges [5]. This is both an opportunity and a warning: centrality captures importance, but importance is not the same as risk.

Lorenz et al. study money-laundering detection under label scarcity and show that graph-centrality features can improve classifiers when labels are limited [6]. Their results support using centrality as a supporting signal rather than a standalone decision rule. ChainLens R4 follows this position. It computes value-weighted PageRank with $\alpha = 0.85$, converts the percentile rank above the 90th percentile into a continuous strength, and discards contributions below 0.05. The 95th percentile is retained as a diagnostic top-five-percent marker. Known exchange, bridge, mixer, and contract addresses are excluded before flagging, because a high PageRank for a router or an exchange is expected and should not be treated as evidence of wrongdoing.

The continuous formulation is a deliberate correction to a binary top-five-percent gate. A wallet at the 96th percentile and a wallet at the 99.9th percentile are not equally central, and a fixed cutoff makes the rule sensitive to cohort composition. The continuous tail preserves the ordering while still limiting R4 to a maximum of 15 points. In the reported run, R4 was the top driver for 7,443 wallets, but most of those contributions were small; only a combination with another rule could reach the Medium tier.

2.2.5 Association Rules, Density Clustering, and Communities

Agrawal and Srikant introduced the Apriori algorithm for mining frequent itemsets and association rules [17]. The framework is attractive for compliance because a rule is expressed as an antecedent–consequent pair with support, confidence, and lift, all of which are readable. ChainLens R5 binarises each wallet into nine behavioural items, mines rules on the training split with the consequent "flagged", and keeps only rules with confidence at least 0.80 and lift at least 2.0. The rule is the most novel part of the design: the items are domain-motivated, but the specific combination of wallet features and association mining is not taken directly from a single paper.

Ester et al. introduced DBSCAN, which finds dense regions without requiring a predefined number of clusters and labels isolated points as noise [18]. ChainLens uses DBSCAN on five scaled features to produce a noise flag, which then becomes an item for R5. DBSCAN does not score wallets directly. This separation is important: density outliers are not automatically suspicious, but the fact that a wallet is an outlier can be useful in combination with other behavioural items. Ostapowicz and Żbikowski show that account-level features such as transaction counts and counterparties are informative for supervised fraud detection [19]; Hirshman et al. show that unsupervised methods can isolate anomalous behaviour in Bitcoin networks [20]. ChainLens borrows the feature space and the outlier intuition, not the supervised labels.

Blondel et al. introduced the Louvain method for modularity maximisation [21]. Phetsouvanh et al. apply graph exploration and community clustering to extortion networks in Bitcoin [22]. ChainLens uses Louvain on the undirected, value-weighted transaction graph as a display-only signal. Community membership helps an analyst see whether a flagged wallet sits inside a larger cluster, but it does not add points. This is a defensible division of labour: community detection is good at describing structure and poor at establishing individual culpability.

Table 2.1 summarises the representative works and the specific gap that ChainLens addresses.

Table 2.1: Representative works and the design gap addressed by ChainLens.

Work	Focus and method	Gap or limitation carried into this project
Victor and Weintraud [12]	SCC-based cycle search for wash trading on DEXs; two-account self-trades; volume ratio	Strong structural method, but a binary membership flag is less informative than a volume-weighted measure; ChainLens reports cycle evidence and uses a conservative $\pm 2\%$ tolerance.
Starnini et al. [14]	Temporal scatter–gather smurf motif with ordered joins	Requires relative velocity calibration; ChainLens uses absolute gates and a 72-hour window, so it is a bounded approximation rather than a full reproduction.
Liu et al. [15]	OFAC sanctions, impurity scoring, dusting resilience	Quantitative exposure and frozen seeds are adopted, but two-hop decay and cross-chain sanctions are not implemented.
Lin et al. [5]	Value-weighted PageRank on Ethereum; heavy-tailed centrality	High centrality includes benign hubs; ChainLens uses a continuous tail and excludes known exchange, bridge, mixer, and contract categories.
Weber et al. [4]	Graph convolutional networks on the Elliptic benchmark	Strong benchmark performance but limited per-wallet explanation; ChainLens keeps a rule vocabulary and an evidence trace instead of a learned embedding.

2.3 RESEARCH GAPS

Five gaps emerge from the review. First, operational systems optimise for coverage and proprietary data, while academic systems optimise for benchmark accuracy; neither necessarily produces a decision that an analyst can defend line by line. Second, rule-based systems often implement only one half of a published typology, such as fan-out without the gather phase. Third, sanctions rules are frequently coupled to the train/test split or ignore dusting, which weakens both validity and robustness. Fourth, centrality rules often use a binary cutoff and do not exclude benign high-volume hubs. Fifth, evaluation is often reported on a balanced sample without stating the base-rate caveat, which makes precision and recall look more

operational than they are.

ChainLens addresses these gaps in a bounded way. It keeps the rule vocabulary small and evidence-bearing, implements the gather phase of the smurf motif, freezes sanctions seeds outside the split, uses a continuous PageRank tail with hub exclusion, and reports the case-control nature of the evaluation. It does not claim to solve the general AML problem. The contribution is a reproducible, inspectable baseline that makes the trade-offs visible.

2.4 SUMMARY

The literature supports the individual ingredients of ChainLens: SCC-based cycle detection, temporal scatter–gather motifs, sanctions exposure with dusting defences, value-weighted centrality, association-rule mining, density clustering, and modularity-based community detection. It also exposes the limits of each ingredient. The next chapter describes the data and preprocessing needed to apply these ideas to a real Ethereum cohort, including the label composition and the leakage-safe split that determine what the evaluation can and cannot claim.

CHAPTER 3

DATASET AND PREPROCESSING

3.1 DATASET DESCRIPTION

The reported experiment uses an artifact-backed real-mode run completed on 28 October 2026. Transaction data was obtained from the Etherscan V2 API for Ethereum mainnet (chain identifier 1) [7]. The extraction uses a per-address recent window: for each address in the planned cohort, up to 500 of its most recent transactions are requested and cached locally. This is not a single contiguous global block interval. The union of fetched records spans block heights from 74,532 to 26,287,961 because different addresses have different activity histories. Twenty additional block heights were sampled as context anchors so that the graph contains addresses that are topologically adjacent to the labelled cohort without being treated as labelled examples.

The run used the local cache: 3,770 Etherscan calls and 1,047 cache hits are recorded in the provenance file, and the fetch stage was marked as skipped because the required cache entries were already present. Caching matters for reproducibility. It reduces API pressure, but it also means that the exact transaction set is frozen at the time of the run. A later re-run against a refreshed cache would produce a different graph even with the same code and configuration.

Table 3.1 summarises the scale of the run. The raw record count is 379,023; after cleaning, 368,113 transactions remain. Feature engineering produces one row per address in the sender/receiver universe, giving 81,756 wallets. The aggregated directed graph contains 99,706 edges. The label store contains 5,446 rows, which collapse to 4,817 unique addresses after priority deduplication. Of these, 2,317 are positive (sanctioned, fraud, or darklist), 2,000 are negative, and 500 are unlabelled context addresses.

Table 3.1: Scale and composition of the reported real-mode run.

Quantity	Value	Note
Raw transaction records	379,023	before cleaning
Clean transactions	368,113	after validation and deduplication
Wallets with engineered features	81,756	union of senders and receivers
Directed graph edges	99,706	aggregated sender–receiver pairs
Label rows	5,446	multiple sources per address possible
Unique labelled or context addresses	4,817	after priority deduplication
Positive addresses	2,317	sanctioned, fraud, or darklist
Negative addresses	2,000	Kaggle negatives
Context addresses	500	unlabelled graph context
Known addresses in cohort	13	downweighted by factor 0.3
Frozen sanctions seeds	120	OFAC snapshot plus labelled seeds
Run duration	50 seconds	01:58:11 to 01:59:01 UTC

3.2 DATA SOURCES AND LABELLING

Three label sources are combined. The first is a frozen OFAC digital-currency address snapshot with 120 Ethereum addresses [3]. The second is a community darklist of malicious Ethereum addresses. The third is a public Kaggle Ethereum fraud dataset that supplies both positive "fraud" addresses and a pool of negative addresses. The label frame records the source, a confidence value, and an as-of date for each row. The OFAC snapshot is dated 30 September 2026; the darklist snapshot is dated 18 November 2020; the Kaggle publication date is 3 January 2021. These dates are not harmonised, and the age of the darklist and Kaggle labels is a real limitation: an address labelled negative in 2021 may have been involved in illicit activity later.

Labels are deduplicated by address using a priority order: sanctioned first, then fraud, then darklist, then negative. This is why 5,446 rows collapse to 4,817 addresses. The priority rule ensures that an address that appears in more than one source is treated as positive if any source marks it positive. Context addresses are kept separate. They are not used as training

labels; they exist to give the graph enough neighbourhood structure for cycle, fan-out, and community detection to operate.

The negative sample is capped at 2,000 addresses, sampled with a fixed seed from the larger Kaggle negative pool. This cap keeps the labelled cohort balanced enough for evaluation while limiting the cost of fetching histories. It also means that the class distribution in the labelled set is not the class distribution of Ethereum. The evaluation is therefore a case-control analysis, not a prevalence estimate.

3.3 DATA CLEANING AND DEDUPLICATION

All addresses are normalised to lowercase and validated against the 40-hex-character EVM pattern. Records with malformed addresses are dropped. Self-transfers, where the sender and receiver are the same address, are removed because they do not represent a transfer of value between entities. Reverted transactions are excluded by requiring a successful receipt status and no error flag. These filters remove failed attempts, contract calls that did not transfer value, and other records that would inflate the graph without carrying economic meaning.

Transactions between two tracked cohort wallets can appear twice: once in the sender's history and once in the recipient's history. The pipeline deduplicates on transaction hash, keeping one row per hash ordered by block number. This is necessary for correct edge weights and for cycle detection; a duplicated edge would double-count value and could create a spurious length-two cycle. After deduplication, 368,113 clean transactions remain from 379,023 raw records, a reduction of 10,910 records or about 2.9%.

Value conversion is handled in DuckDB using fixed-point arithmetic. The raw value is stored as a wei string and converted to a `DECIMAL(38,18)` Ether value. The conversion preserves exact 18-decimal precision:

$$\text{value_eth} = \left\lfloor \frac{\text{value_wei}}{10^{18}} \right\rfloor + \frac{\text{value_wei} \bmod 10^{18}}{10^{18}}. \tag{3.1}$$

Using floating-point arithmetic for this step would lose precision on large balances and could make two economically equal transfers differ by a rounding artefact. The use of a fixed-point type is also why the feature layer can safely compare transfer values within a $\pm 2\%$ tolerance.

3.4 FEATURE ENGINEERING

The feature layer is a single DuckDB query over the clean transaction table. It builds a universe of all senders and receivers, then computes outbound, inbound, activity, and contract indicators for each address. Fourteen behavioural features are produced; they are listed in Table 3.2. The features are intentionally simple and interpretable. Each one corresponds to a question an analyst might ask: how active is the wallet, how dispersed are its outbound transfers, how much value does it move, and does it look like a contract?

Table 3.2: The fourteen behavioural features engineered for each wallet.

Feature	**Definition**	**Forensic interpretation**
`tx_count_out`	Number of outbound transfers	Outbound activity
`tx_count_in`	Number of inbound transfers	Inbound funding activity
`tx_count_total`	Sum of inbound and outbound counts	Gross transaction velocity
`unique_counterparties`	Distinct outbound recipients	Dispersion of funds
`unique_senders`	Distinct inbound senders	Aggregation of funds
`fanout_ratio`	Unique counterparties divided by outbound count	Non-repeating dispersion
`out_in_ratio`	Outbound count divided by inbound count	Draining versus accumulating behaviour
`total_value_eth`	Sum of outbound Ether value	Gross outbound volume
`avg_value_eth`	Mean outbound transfer value	Average payment size
`median_value_eth`	Median outbound transfer value	Outlier-resistant typical size
`net_flow_eth`	Inbound value minus outbound value	Capital retention or liquidation
`is_contract`	Whether the address created a contract	Entity type indicator
`active_days`	Distinct calendar days with activity	Temporal persistence
`round_number_ratio`	Fraction of outbound values equal to a whole number	Human or algorithmic payment pattern

Two features deserve comment. The fan-out ratio is defined as unique counterparties

divided by outbound transaction count. A value near 1.0 means that the wallet almost never sends twice to the same address, which is the dispersion behaviour that the smurf rule expects. A value near 0 means that the wallet repeatedly transacts with a small set of counterparties, which is common for operational wallets. The round-number ratio captures the fraction of transfers whose value has no fractional part. Whole-ETH payments are common in ordinary commerce, so this feature is not suspicious on its own; it becomes useful only in combination with other items in the Apriori profile.

The feature table also records first and last seen timestamps and the number of active days. These are used for the recent-account item in R5 and for descriptive analysis, but they are not all treated as model inputs. The five features used for scaling and DBSCAN are `total_value_eth`, `tx_count_out`, `tx_count_in`, `unique_counterparties`, and `fanout_ratio`. The remaining features are used directly by rules or by the association-rule item set.

3.5 LABELLED COHORT AND TRAIN/TEST SPLIT

The labelled cohort is split at the address level so that no wallet appears in both training and test. The implementation uses scikit-learn's `train_test_split` with a fixed random seed of 42 and a test size of 0.3, stratified by label group (positive versus negative) where both groups have at least two examples. This is the code path used by `fetch_data.py`. An earlier design note described a SHA-256 hash-modulo split; that description does not match the implemented path and is not used in this report.

The split produces 3,021 training addresses and 1,296 test addresses, with 500 context addresses held outside the labelled split. After joining features, 158 positive and 1 negative test addresses are dropped because they have no activity in the fetched window. The evaluation therefore runs on 538 positives and 599 negatives, a positive rate of 47.3%. The training side retains 1,621 positives and 1,400 negatives. Table 3.3 summarises the composition.

Table 3.3: Label composition and leakage-safe address split.

Partition	Positive	Negative	Total	Use
Training	1,621	1,400	3,021	scaler fitting, Apriori mining
Test (before activity join)	696	600	1,296	held-out evaluation
Test (after activity join)	538	599	1,137	reported metrics
Dropped for no activity	158	1	159	excluded from metrics
Context (unlabelled)	–	–	500	graph context only

The case-control design is central to interpreting the results. Because negatives were sampled to roughly match positives, the test set has a positive rate near one half. In the full Ethereum population, confirmed illicit addresses are far rarer. Precision measured on this sample is therefore not the precision an operational team would see at the same threshold. The report states this caveat wherever metrics are presented.

3.6 FEATURE SCALING AND LEAKAGE CONTROL

Five continuous features are transformed before DBSCAN and before any model that assumes a common scale. Features with strong positive skew are log-transformed as $\ln(1+x)$ and then standardised using the mean and standard deviation of the training partition:

$$z = \frac{\ln(1+x) - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{3.2}$$

The scaler is fitted only on training addresses. Test addresses are transformed with the frozen training parameters. This prevents information from the test distribution from influencing the feature space. The scaler parameters are hashed and recorded in the run provenance; the reported run uses scaler version `4a91e3055783`.

Leakage control also applies to R5. Apriori itemsets are mined on the training split only. The test split is never used to choose items, thresholds, or rules. R3 is the exception by design: its sanctions seeds are a frozen external list unioned with labelled sanctioned addresses, and split membership is deliberately irrelevant. This is not leakage in the sense of using test labels to fit the model; it is a policy choice that reflects how sanctions compliance actually works.

A sanctioned address is sanctioned regardless of which partition it falls into.

3.7 SUMMARY

The dataset chapter establishes both the scale and the boundaries of the evidence. The run has 81,756 wallets, 99,706 graph edges, 4,817 unique labelled or context addresses, and 1,137 active test addresses. Cleaning removes reverted, self-referential, malformed, and duplicate records. Feature engineering produces 14 interpretable measures with fixed-point value arithmetic. The split is seeded, stratified, and address-level, with scaling and association mining restricted to the training partition. The next chapter describes how these inputs are converted into patterns, scores, and an audit trail.

CHAPTER 4

PROPOSED SYSTEM AND METHODOLOGY

4.1 SYSTEM OVERVIEW AND ARCHITECTURE

ChainLens is organised as a decoupled batch pipeline with a read-only presentation layer. The architecture is shown in Figure 4.1. External sources—the Etherscan API, the label store, a frozen OFAC snapshot, and a known-entity registry—feed an ingestion stage that normalises addresses, caches transactions, and assembles the labelled cohort. The feature stage uses DuckDB to clean, deduplicate, and aggregate transactions into one row per wallet. The detection stage builds a directed transaction graph and runs five rules plus DBSCAN and Louvain. The scoring stage converts rule evidence into strengths, applies locked weights and known-entity downweighting, clamps the result, and writes a per-wallet trace. The evaluation stage computes operating-point metrics and a leave-one-rule-out ablation.

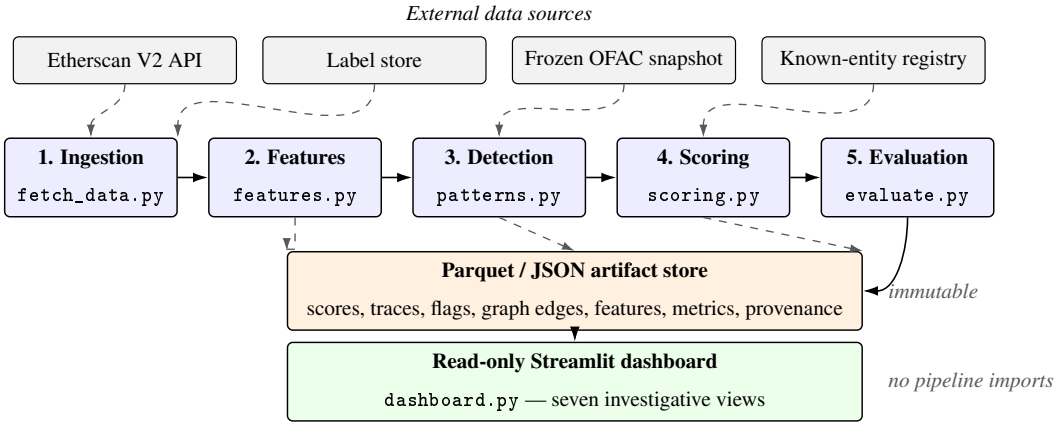


Figure 4.1: Decoupled architecture of ChainLens. External sources feed a five-stage batch pipeline; every stage writes immutable Parquet or JSON artifacts that the read-only dashboard consumes without importing pipeline code.

The most important architectural decision is the separation between computation and presentation. The dashboard does not import `features.py`, `patterns.py`, or `scoring.py`. It reads Parquet and JSON artifacts from `data/output/` and renders them. This has two consequences. First, the dashboard cannot accidentally mutate a score or trigger a network

call. Second, the artifacts are the unit of audit: an analyst can compare the dashboard view with the stored trace, and a reviewer can re-run the evaluation without opening the user interface. The same separation makes the dashboard fast, because it never recomputes a graph algorithm while a user is waiting.

A second decision is the locked rule contract. The weights are stored in `config/rules.yaml`, validated at startup, and hashed. The reported run records rules hash `4cd18b0a9038adb3a8eb1261676eb797ab2fb3d90c8a315438817829fa26164e` and rules version label `v0-deck`. A change to a weight or threshold produces a different hash and a different run identity. This does not make the weights objectively correct, but it makes drift visible. In a compliance setting, the ability to prove that the same contract was used is often as important as the contract itself.

4.2 THEORETICAL BACKGROUND

4.2.1 Graph Representation

Let $G = (V, E)$ be a directed graph in which each node $a \in V$ is an Ethereum address and each edge $(u, v) \in E$ represents one or more transfers from u to v. An edge carries a transaction count, a total value in Ether, and the timestamps of the underlying transfers. The graph is a simplification: it aggregates multiple transactions between the same pair and ignores internal contract calls that are not visible as top-level transfers. This simplification is necessary for tractability and is consistent with the evidence views in the dashboard, which show aggregated edges and list the underlying transactions separately.

For a node a, let $N^+(a)$ be its out-neighbours and $N^-(a)$ its in-neighbours. The out-degree is $|N^+(a)|$, and the weighted out-strength is the sum of edge values leaving a. The feature layer computes both counts and values. The graph layer uses the same edge set for cycle detection, fan-out, sanctions proximity, PageRank, and community detection, so a single cleaning decision propagates consistently through all rules.

4.2.2 Evidence Strength and Composite Score

Each rule i produces a strength $s_i(a) \in [0, 1]$. R1, R2, and R5 are binary: a wallet either satisfies the rule or it does not, so s_i is 1 or 0. R3 and R4 are continuous. R3 uses the larger

of the inflow and outflow exposure fractions, with a floor of 0.10 for a confirmed direct link. R4 uses the position of the wallet in the PageRank percentile distribution above the 90th percentile. The raw score is the weighted sum

$$r(a) = \sum_{i=1}^{5} w_i s_i(a), \qquad w = (25, 20, 30, 15, 20). \tag{4.1}$$

The maximum raw score is 110, but the final score is clamped to 100. If the wallet belongs to a known benign category $K = \{\text{exchange}, \text{bridge}, \text{mixer}, \text{contract}\}$, the raw score is multiplied by 0.3:

$$\tilde{r}(a) = \begin{cases} 0.3\, r(a), & a \in K, \\ r(a), & \text{otherwise}, \end{cases} \qquad \text{score}(a) = \min(100, \max(0, \text{round}(\tilde{r}(a)))). \tag{4.2}$$

The downweight is a blunt instrument. It suppresses exchanges and routers that would otherwise dominate the high end of the distribution, but it can also suppress a genuinely risky address that happens to be registered as a contract. The trace records the category and the pre- and post-downweight values so that the decision can be reviewed. In the reported run, 13 cohort addresses were downweighted; the highest raw score among them was 26.84, which became 8 after the 0.3 factor.

4.2.3 Explainability and Provenance

Each wallet receives a trace with a fixed schema. The trace records the rule contract version, scaler version, evaluation timestamp, raw score, adjusted raw score, final score, tier, top driver, clamp status, downweight details, and a per-rule object. Each per-rule object contains the rule identifier, name, hit flag, weight, strength, contribution, and an evidence payload when the rule fired. For R3 and R4, a gate message is recorded when the wallet was seen by the rule but the strength fell below the minimum. This makes a non-hit informative: an analyst can distinguish “no sanctioned link” from “a link existed but its exposure was too small to score”.

The trace is validated before it is persisted. Validation checks that contributions sum to the raw score within tolerance, that the tier is consistent with the final score, that the downweight arithmetic is consistent, and that the final score is inside the allowed range. A

trace that fails validation is a bug, not a modelling choice. The dashboard reads the same trace and renders the evidence cards, the contribution bars, and the raw JSON. This is the practical meaning of explainability in ChainLens: not a post-hoc saliency map, but a stored decision record that can be inspected independently of the code that produced it.

4.3 DATA MINING ALGORITHMS

4.3.1 R1: Circular Flow Detection

R1 looks for directed cycles of length two to five in which value returns to its origin within a bounded time. The algorithm first computes strongly connected components (SCCs) of G. A cycle can only exist inside an SCC, so this step removes the acyclic majority of the graph. SCCs larger than 5,000 nodes are skipped to avoid combinatorial explosion. Within each SCC, the implementation enumerates simple cycles, canonicalises them to avoid rotation duplicates, and keeps at most five cycles per address.

A candidate cycle $C = (v_0, v_1, \ldots, v_{k-1}, v_0)$ must satisfy three conditions. The length k must be between two and five. The transfer values must be approximately equal:

$$\frac{\max_{e \in C} v_e - \min_{e \in C} v_e}{\max_{e \in C} v_e} \leq 0.02. \tag{4.3}$$

The time between the first and last transfer must not exceed 24 hours. The $\pm 2\%$ tolerance is tighter than the $\pm 5\%$ used in an earlier version and is intended to reflect gas-fee noise rather than genuine variation in transfer size. A length-two cycle with values 10.0 and 10.0 ETH, separated by a few hours, is the canonical case; a length-two cycle with values 10.0 and 10.5 ETH is not.

R1 is a high-precision, low-recall rule. It cannot see cycles that pass through addresses outside the fetched cohort, and it deliberately excludes long cycles that are common in ordinary routing. In the reported run, R1 found 97 qualifying length-two cycles involving 163 unique wallets. No longer cycles qualified under the value and time constraints. The evidence payload records the cycle path, edge values, maximum-to-minimum ratio, time span, and SCC size.

4.3.2 R2: Scatter–Gather Smurfing

R2 implements the two-part motif described by Starnini et al. [14]. The scatter gate requires at least 20 unique counterparties, at least 20 outbound transactions, and a fan-out ratio of at least 0.5. These thresholds identify wallets that disperse funds rather than repeatedly paying the same recipients. The gather gate then examines the out-neighbours of the candidate. At least two intermediaries, or 10% of the scattering counterparties, must forward funds onward; at least one distinct collector must receive those forwards; and each gather edge must occur after its scatter edge and within 72 hours.

The temporal condition is the part that distinguishes R2 from a simple fan-out count. A wallet that sends to 30 addresses and stops is a candidate, not a confirmed smurf. A wallet whose intermediaries forward funds to a common collector shortly afterwards is a much stronger signal. The implementation records the number of unique counterparties, outbound transactions, fan-out ratio, gather intermediaries, collectors, gather fraction, and the temporal ordering in the evidence payload. If the scatter gate passes but the gather gate fails, the wallet receives no R2 points.

R2 is deliberately conservative. The 20-counterparty and 20-transaction thresholds are absolute, not relative to the cohort's transaction velocity. A small cohort may therefore produce no R2 hits even when smurf-like behaviour exists. The reported run produced 33 R2 members, down from 240 in an earlier fan-out-only version. The reduction is expected: the earlier version counted dispersion, while the current version requires the gather continuation.

4.3.3 R3: Sanctioned One-Hop Exposure

R3 flags a wallet that directly sent to or received from a frozen sanctions seed, provided the transfer value is at least 0.01 ETH. Seeds are the union of the 120-address OFAC snapshot and any labelled sanctioned address. The rule does not depend on train/test membership. It examines immediate predecessors and successors only; two-hop exposure is not scored. This is a deliberate scope restriction. One hop is the strongest and most defensible relationship, while multi-hop propagation produces a large and difficult-to-calibrate false-positive surface.

For each flagged wallet, the rule computes an inflow exposure fraction and an outflow

exposure fraction. The strength is

$$s_3(a) = \min(1, \max(\text{exposure}(a), 0.10)), \tag{4.4}$$

where exposure(a) is the larger of the inflow and outflow fractions. The 0.10 floor ensures that a confirmed direct link always contributes at least 3 of the 30 available points, even if the sanctioned amount is tiny relative to the wallet's total volume. Transfers below 0.01 ETH are ignored to resist dusting attacks, in which an adversary sends a negligible amount to an innocent wallet in order to create a misleading link.

The evidence payload lists each seed, the direction of the relationship, the ETH amount, and the exposure fractions. In the reported run, R3 flagged 2,014 wallets with a mean strength of 0.963. The high mean indicates that most flagged wallets had a large fraction of their volume exposed to a seed, not merely a marginal link. R3 was also the most common top driver among scored wallets, which is consistent with its 30-point weight.

4.3.4 R4: Continuous Value-Weighted PageRank

PageRank assigns importance to a node based on the importance of its predecessors [16]. ChainLens uses a value-weighted variant:

$$PR(u) = \frac{1-\alpha}{|V|} + \alpha \sum_{v \in M(u)} \frac{PR(v)\, w(v,u)}{\sum_{k \in N^+(v)} w(v,k)}, \tag{4.5}$$

where $\alpha = 0.85$, $M(u)$ is the set of in-neighbours of u, and $w(v,u)$ is the total value transferred from v to u. The implementation uses NetworkX's PageRank with a convergence tolerance of 10^{-6}. Edge weights are floored at 10^{-12} so that a zero-value edge does not create a division by zero.

R4 converts the PageRank percentile $\pi(u)$ into a continuous strength:

$$s_4(u) = \min\left(1, \max\left(0, \frac{\pi(u) - 90}{10}\right)\right). \tag{4.6}$$

A wallet at the 90th percentile receives no points; a wallet at the 95th percentile receives half of the 15-point weight; a wallet at or above the 100th percentile receives the full weight. Contributions below 0.05 strength are discarded. The 95th percentile is retained as a

diagnostic top-five-percent marker but is not used as a hard gate. Known exchange, bridge, mixer, and contract addresses are excluded before flagging, because a high PageRank for a router or an exchange is expected. In the reported run, 7,758 wallets were in the continuous tail and 4,100 were in the diagnostic top five percent; 15 known-category addresses were excluded from R4 flagging.

4.3.5 R5: Apriori Association Rules

R5 treats each wallet as a transaction basket. Nine boolean items are defined: `high_fanout` (at least 20 unique counterparties), `high_tx_count` (at least 10 total transactions), `low_value` (median transfer below 0.1 ETH), `high_value` (total value at least 10 ETH), `many_round` (at least half of transfers are whole-ETH amounts), `is_contract`, `recent_account` (first seen within seven days of the window end), `high_pagerank` (in the diagnostic top five percent), and `dbscan_noise`. The items are not a model of all suspicious behaviour. They are a compact vocabulary that captures dispersion, value, contract status, recency, centrality, and outlier status.

Apriori mines frequent itemsets on the training split [17]. For an itemset X and a wallet population A,

$$\text{supp}(X) = \frac{|\{a \in A : X \subseteq I(a)\}|}{|A|}, \tag{4.7}$$

where $I(a)$ is the item set of wallet a. Association rules $X \Rightarrow Y$ are ranked by confidence and lift:

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}, \qquad \text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)}. \tag{4.8}$$

The implementation uses a minimum support of 0.05, a minimum confidence of 0.80, a minimum lift of 2.0, and a maximum antecedent size of 3. The consequent is the labelled "flagged" item. A wallet that satisfies the antecedent of a kept rule receives R5 points.

R5 is the most exploratory rule in the design. The item set is domain-motivated but not taken directly from a single paper, and the rule can only work when the training split contains enough labelled positives to support stable itemsets. In the reported run it produced no rules: the best candidate lift was 1.394, below the locked threshold of 2.0. The rule therefore contributed zero points, and the effective system was R1–R4. This is reported as a result, not hidden as a configuration detail.

4.3.6 DBSCAN Density Clustering

DBSCAN finds dense regions in feature space and labels isolated points as noise [18]. ChainLens applies DBSCAN to five scaled features—total value, outbound count, inbound count, unique counterparties, and fan-out ratio—with $\varepsilon = 1.0122$ and min_samples $= 5$. The epsilon value was derived from the 95th percentile of the five-nearest-neighbour distance in the scaled space. Wallets labelled -1 receive the `dbscan_noise` flag. Wallets in a dense region receive a cluster label, which is displayed in the dashboard but is not scored.

DBSCAN does not add points directly. Its role is to supply the `dbscan_noise` item to R5 and to give the analyst a way to see whether a wallet is an outlier relative to the cohort's behavioural distribution. In the reported run, DBSCAN produced three clusters and 34 noise points. The small number of clusters is a consequence of the strong skew in wallet activity: most wallets sit near zero, and the dense region is dominated by low-activity addresses.

4.3.7 Louvain Community Detection

Louvain maximises modularity on an undirected weighted graph [21]. For a partition of nodes into communities c_i, modularity is

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{4.9}$$

where A_{ij} is the edge weight, k_i is the weighted degree, m is the total edge weight, and δ is 1 when two nodes are in the same community. ChainLens runs Louvain on the undirected version of the transaction graph, weighted by total value, with resolution 1.0 and a fixed seed. Communities are numbered stably by sorting on the smallest member address.

Louvain is display-only. It does not contribute to the score. Its value is investigative: if a flagged wallet belongs to a community with several other flagged wallets, the analyst can inspect the community as a cluster rather than as an isolated address. In the reported run, Louvain found 18,512 communities across 81,756 wallets. The large number reflects the fragmented nature of the fetched cohort: many addresses have only a few edges, and the graph does not form a single connected component.

4.4 COMPOSITE SCORING AND RISK TIERING

The locked rule contract is summarised in Table 4.1. The weights are not learned from the data. They encode a severity ordering in which direct sanctions exposure (30) is the strongest signal, circular flow (25) is next, fan-out and association matching (20 each) are supporting signals, and PageRank (15) is the weakest. The ordering is consistent with the AML literature's emphasis on direct illicit exposure and with the operational principle that a sanctions link is more actionable than a centrality rank.

Table 4.1: Locked rule weights and principal parameters.

Rule	Signal	Weight	Principal parameters
R1	Circular flow	25	cycle length 2–5; value tolerance $\pm 2\%$; 24-hour window; at most 5 cycles per address; SCC cap 5,000
R2	Scatter–gather smurf	20	at least 20 counterparties, 20 outbound transactions, fan-out ratio 0.5; gather within 72 hours; at least 2 intermediaries or 10%; at least 1 collector
R3	Sanctioned one-hop	30	120 frozen seeds; minimum 0.01 ETH; strength $\max(\text{exposure}, 0.10)$
R4	PageRank tail	15	$\alpha = 0.85$; percentile ≥ 90; strength $(\pi - 90)/10$; minimum 0.05; known hubs excluded
R5	Apriori match	20	support 0.05; confidence 0.80; lift 2.0; antecedent size ≤ 3; train-only

The tier thresholds are given in Table 4.2. The High tier is deliberately hard to reach. With the locked weights, no pair of rules can reach 60: the strongest pair is R3 + R1 = 55, and the strongest pair without R3 is R1 + R2 = 45. A High score therefore requires at least three concurrent signals, or a combination that includes continuous R4 contributions on top of two strong rules. This is an intentional precision mechanism, but it also explains why the High tier is nearly empty on a case-control sample.

Table 4.2: Risk tiers and their interpretation.

Tier	Score range	Operational interpretation
High	60–100	Multiple independent signals; priority forensic review
Medium	25–59	At least one strong signal or a combination of supporting signals; active triage queue
Low	0–24	No signal, or weak/continuous evidence below the triage threshold

The top driver is the rule with the largest contribution. If the raw score is zero, the top driver is empty. The driver is a summary, not a verdict: a wallet with R3 = 30 and R4 = 12 has R3 as its driver, but the R4 contribution may still be the difference between Medium and High. The dashboard therefore shows both the driver and the full contribution breakdown.

4.5 SYSTEM WORKFLOW

Figure 4.2 shows the end-to-end workflow. Acquisition and cleaning produce a transaction table. Feature engineering produces one row per wallet. Labelling and splitting produce train and test partitions. Detection consumes the graph and the scaled features. Scoring consumes the flags and the known-entity registry. Evaluation consumes the scores and the labels. Persistence writes the artifacts that the dashboard reads.

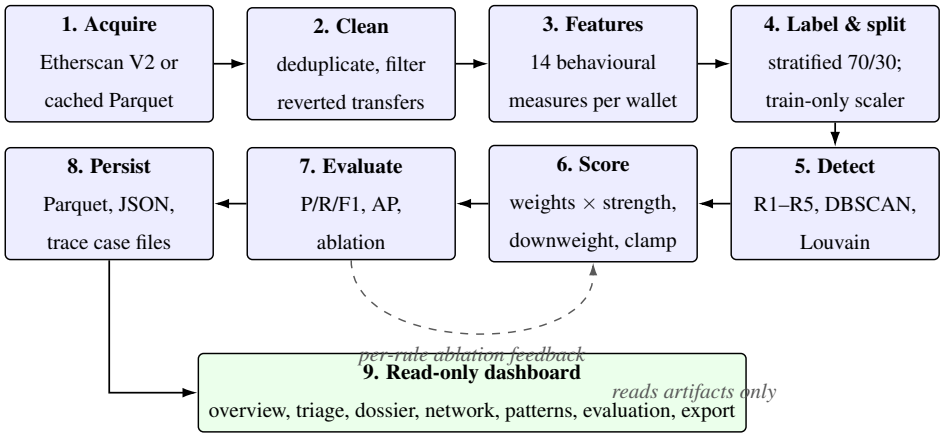


Figure 4.2: End-to-end workflow of the reported run. Solid arrows show the batch path from acquisition to persistence; the dashed arrow marks the leave-one-rule-out feedback used during evaluation. The dashboard is a separate read-only consumer.

The workflow is batch-oriented, but it is not monolithic. Each stage writes a durable artifact, and the next stage can be re-run from that artifact without repeating the previous stage. This is what makes the ablation study feasible: the evaluation stage can re-score the same flags with one rule's weight set to zero, without re-fetching transactions or re-running the graph algorithms. It is also what makes the dashboard trustworthy: the view is a rendering of the same artifacts that the evaluation used.

4.6 EVALUATION METHODOLOGY

The evaluation uses the held-out test partition. For a threshold τ, a wallet is predicted positive if its score is at least τ. The standard metrics are

$$P = \frac{TP}{TP+FP}, \qquad R = \frac{TP}{TP+FN}, \qquad F1 = \frac{2PR}{P+R}. \tag{4.10}$$

Two operating points are reported: the High threshold $\tau = 60$ and the Medium threshold $\tau = 25$. The High threshold is the primary compliance queue; the Medium threshold is the broader triage queue. Average precision (AP) is computed from the precision–recall curve:

$$AP = \sum_{n} (R_n - R_{n-1}) P_n, \tag{4.11}$$

where the sum runs over the ranked score thresholds. AP is threshold-independent and is therefore a useful complement to the two operating points.

The leave-one-rule-out ablation re-scores the cohort with one rule's weight set to zero and recomputes the metrics:

$$\Delta F1_i = F1_{-i} - F1_{\text{base}}. \tag{4.12}$$

A negative $\Delta F1_i$ means that removing rule i reduced F1, so the rule was useful at that operating point. A positive value means that the rule added false positives or removed true positives. The ablation also reports the number of wallets whose tier changed when the rule was removed. This second quantity is important when the baseline F1 is zero: $\Delta F1$ is then degenerate, but tier changes still show which rule is holding the queue together.

All metrics are computed on the case-control test sample. The report does not extrapolate them to the Ethereum population. The positive rate in the sample is 47.3%, while the

operational positive rate is unknown and almost certainly much lower. Precision at a fixed threshold depends strongly on prevalence, so the reported precision is a sample statistic, not an operational guarantee.

4.7 SUMMARY

The methodology combines a small set of graph and behavioural rules with a locked additive score and a trace-first design. R1 detects short value-matched cycles; R2 detects the temporal scatter–gather motif; R3 detects direct sanctions exposure with dusting defences; R4 uses a continuous PageRank tail with hub exclusion; R5 mines association rules on training positives. DBSCAN and Louvain provide non-scoring investigative signals. The score is weighted, downweighted for known benign entities, clamped, and tiered. The evaluation reports two operating points, average precision, and a leave-one-rule-out ablation with tier-change counts. The next chapter applies this methodology to the reported run and examines what the results do and do not support.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 IMPLEMENTATION ENVIRONMENT AND TECHNOLOGY STACK

ChainLens is implemented in Python and organised as a set of single-responsibility modules orchestrated by `run.py`. The reported run was produced from git commit `cf83c1a` with the package versions recorded in the provenance file. Table 5.1 lists the principal technologies and the reason each was selected. The stack is deliberately small: an analytical database, a graph library, two machine-learning libraries, a dataframe layer, and a dashboard framework.

Table 5.1: Implementation stack and rationale.

Layer	**Technology (reported version)**	**Role and rationale**
Language and runtime	Python (3.11+ project environment)	Readable data-processing code with a mature analytics ecosystem.
Ingestion	Etherscan V2 API, `requests`	Public Ethereum mainnet access with a local per-address cache [7].
Analytical storage	DuckDB 2.0.0, Parquet	In-process OLAP with exact `DECIMAL(38,18)` arithmetic and columnar artifacts [9].
Dataframes	pandas 2.3.3, NumPy 2.4.6	Tabular transformations, vectorised scoring, and artifact export.
Graph analytics	NetworkX 3.6.1	Graph construction, SCC, bounded cycles, PageRank, and Louvain [8].
Density clustering	scikit-learn 1.9.1	DBSCAN with a fixed epsilon and minimum sample count [10].
Association rules	MLxtend 0.25.0	Apriori frequent itemsets and association-rule metrics [11].
Dashboard	Streamlit 1.66.1, Plotly	Read-only, seven-view analyst workspace [23].
Quality assurance	pytest, Ruff	Unit and integration tests; linting and formatting.
Provenance	git, SHA-256, JSON	Commit, rule hash, scaler hash, package versions, and timestamps.

5.2 MODULE IMPLEMENTATION

5.2.1 Ingestion

The ingestion module, `fetch_data.py`, implements a rate-limited Etherscan client with per-address Parquet caching. Each address is requested with a bounded recent window of up to 500 transactions. Cached responses are reused when present; the reported run records 3,770 API calls and 1,047 cache hits and marks the fetch stage as skipped because the cache was complete. Addresses are normalised to lowercase, validated against the EVM pattern, and written to a raw transaction table.

The same module loads the label sources, applies the priority order (sanctioned, fraud, darklist, negative), samples the negative pool, and performs the address-level train/test split. The split is seeded and stratified where possible. Context addresses are kept in a separate partition. The module also records the split seed so that the assignment can be reproduced.

5.2.2 Feature Engineering

The feature module, `features.py`, opens a DuckDB connection, registers the raw transactions, removes self-transfers and failed transactions, deduplicates on transaction hash, and converts wei strings to exact Ether decimals. It then executes a single SQL statement that builds the sender/receiver universe and computes the 14 features described in Chapter 3. The result is written as `wallet_features.parquet` and as a DuckDB table.

Scaling is a separate function. It selects the five continuous features, applies a log-plus-one transform where the distribution is positively skewed, fits a standardiser on training addresses only, and writes the transformed columns and the scaler metadata. The scaler hash in the reported run is `4a91e3055783`. The function also writes `transactions_clean.parquet`, which is the evidence table used by the dashboard's transaction view.

5.2.3 Pattern Detection

The detection module, `patterns.py`, builds a directed graph from aggregated edges. Each edge stores the transaction count, total value, first timestamp, and last timestamp. The module then runs the five rules and the two unsupervised methods. R1 uses NetworkX's strongly

connected components and a bounded cycle search. R2 uses the scatter gates and a temporal gather check over the candidate's out-neighbours. R3 checks immediate neighbours against the frozen sanctions set and computes exposure fractions. R4 computes value-weighted PageRank and converts percentiles to strengths. R5 builds the nine-item boolean table, mines Apriori rules on training positives, and matches antecedents across the cohort. DBSCAN and Louvain run on the scaled features and the undirected graph respectively.

All signals are normalised into a long-format flags table with columns for address, signal name, signal value, and evidence JSON. This table is the interface between detection and scoring. It also makes the detection output inspectable without reading the rule code: the evidence column contains the cycle path, the fan-out counts, the sanction seed and amount, the PageRank percentile, or the Apriori antecedent.

5.2.4 Scoring and Traces

The scoring module, `scoring.py`, loads the locked rule configuration and the known-address registry. It converts flags into per-rule strengths, applies the weights, sums the contributions, applies the 0.3 downweight where appropriate, clamps to 100, and assigns a tier. It then writes one score row and one trace row per wallet.

The trace is the core audit object. It contains the rule contract version, scaler version, timestamp, raw and adjusted scores, final score, tier, top driver, clamp status, downweight details, and a per-rule record. Each per-rule record contains the hit flag, weight, strength, contribution, evidence inputs, and a gate message when a rule was evaluated but did not score. The module validates the trace arithmetic before persistence. This validation caught inconsistencies during development and is part of the reason the reported scores can be reconstructed from the stored artifacts.

5.2.5 Evaluation

The evaluation module, `evaluate.py`, joins scores to labels and splits, drops test addresses with no activity, and computes precision, recall, F1, average precision, and confusion matrices at the High and Medium thresholds. It then performs the leave-one-rule-out ablation: for each rule, it sets the weight to zero, re-scores the cohort, recomputes the High-threshold metrics,

and records the number of wallets whose tier changed. The outputs are `evaluation.json`, `evaluation.md`, and `ablation.csv`.

The ablation is intentionally simple. It does not retrain a model, because there is no learned model to retrain. It asks a counterfactual question about the locked scoring contract: what would the queue look like if this rule were removed? That question is answerable from the stored flags, which is why the ablation can be reproduced without repeating the graph analysis.

5.3 DASHBOARD IMPLEMENTATION

The dashboard is a Streamlit application with seven views: Overview, Triage, Wallet Dossier, Network, Patterns, Evaluation, and Data & Export. It is strictly read-only. It reads the Parquet and JSON artifacts, caches them for the session, and renders the results. It does not import the pipeline modules and does not write to the artifact directory.

The Overview view combines six headline metrics with a risk-score histogram and a signal-frequency chart. The Triage view provides filters for tier, score range, triggered rule, split, known category, address or label text, flagged-only, and watchlist-only. The Dossier view shows five rule cards, a contribution breakdown, and sub-tabs for transactions, behavioural features, graph neighbours, and the raw case file. The Network view renders an ego-subgraph with controls for focal address, hop count, node cap, and colour mode. The Patterns view exposes the raw output of each rule and of DBSCAN and Louvain. The Evaluation view shows the two operating points, confusion matrices, and the ablation table. The Data & Export view lists the artifact inventory, the provenance JSON, and a download bundle.

The dashboard's visual language is intentionally plain. Risk tiers use a small palette (red, amber, slate), rule evidence uses a consistent card layout, and the raw JSON is always one click away. The design goal is not to persuade the analyst that the score is correct; it is to make the basis of the score easy to inspect and easy to challenge.

5.4 RUN SUMMARY

Table 5.2 records the provenance of the reported run. The run identifier encodes the start timestamp and a short hash. The mode is `real`, the seed is 42, and the chain identifier is 1. The run completed in 50 seconds. The rules hash and scaler hash are the two most important reproducibility anchors: they identify the exact rule contract and the exact feature transformation used to produce the scores.

Table 5.2: Provenance of the reported run (28 October 2026).

Field	Value
Run identifier	`2026-10-28T01:58:11Z-786fffd57a92`
Mode and source	`real`; `etherscan_v2`; chain 1
Seed	42
Window	per-address recent; at most 500 transactions per address
Context blocks	20 sampled heights
Etherscan calls / cache hits	3,770 / 1,047
Duration	50 seconds (01:58:11–01:59:01 UTC)
Git commit	`cf83c1a`
Rules version / hash	`v0-deck` / `4cd18b0a9038adb3a8eb1261676eb797ab2fb3d90c8a315438817829fa26164e`
Scaler version	`4a91e3055783`
Package versions	pandas 2.3.3; NumPy 2.4.6; DuckDB 2.0.0; NetworkX 3.6.1; scikit-learn 1.9.1; MLxtend 0.25.0; Streamlit 1.66.1

5.5 PATTERN DETECTION RESULTS

Table 5.3 summarises the detection output. The five rules produced very different volumes. R1 found 97 length-two cycles involving 163 unique wallets. No length-three, length-four, or length-five cycle passed the value and time gates. R2 confirmed 33 scatter–gather wallets. R3 flagged 2,014 wallets with a mean strength of 0.963. R4 placed 7,758 wallets in the continuous PageRank tail, of which 4,100 were also in the diagnostic top five percent. R5

produced no rules because the best candidate lift was 1.394, below the locked minimum of 2.0. DBSCAN produced three clusters and 34 noise points. Louvain produced 18,512 communities.

Table 5.3: Pattern detection output for the reported run.

Signal	Method	Wallets	Observation
R1	SCC + bounded cycles, 2–5, $\pm 2\%$, 24 h	163	97 qualifying cycles; all length 2
R2	Scatter gates + 72 h gather ordering	33	scatter-only candidates excluded
R3	Frozen sanctions, one hop, ≥ 0.01 ETH	2,014	mean evidence strength 0.963
R4	Value-weighted PageRank, continuous tail	7,758	4,100 diagnostic top 5%; 15 hubs excluded
R5	Apriori on nine boolean items	0	best lift $1.394 < 2.0$
DBSCAN	Density clustering, $\varepsilon = 1.0122$, min 5	34 noise	3 dense clusters
Louvain	Modularity on undirected value graph	81,756	18,512 communities; display-only
Union	Rule or DBSCAN signals	9,627	investigative queue
Scored	Final score > 0	9,621	six DBSCAN-only wallets score 0

The difference between 9,627 flagged wallets and 9,621 scored wallets is small but worth explaining. The flagged count includes wallets with at least one investigative signal other than Louvain, including DBSCAN noise. DBSCAN noise does not add points by itself. Six wallets were DBSCAN noise but had no scoring rule hit, so they appear in the investigative queue with a score of zero. The dashboard's "Flagged Wallets" card therefore reports 9,627, while the score distribution has 9,621 positive values.

R1's result is striking because all 97 cycles are length two. This is consistent with the literature's emphasis on two-account self-trade pairs and with the $\pm 2\%$ tolerance: longer

cycles are more likely to contain value variation that exceeds the tolerance. It also reflects the bounded cohort. A cycle that passes through an address outside the fetched set cannot be seen, and the graph is a subgraph of the full Ethereum transaction network.

R5's zero result is not a failure of the implementation. The best candidate rule had a lift of 1.394, which is well below the 2.0 threshold, and the training split did not contain a sufficiently distinctive combination of items to support a high-confidence rule. Lowering the lift threshold would produce rules, but those rules would be weaker than the design contract allows. The correct response is to report the zero and treat R5 as inactive on this dataset, not to tune the threshold after seeing the test result.

5.6 RISK SCORING RESULTS

Table 5.4 shows the final tier distribution. There is one High-tier wallet, 2,094 Medium-tier wallets, and 79,661 Low-tier wallets. The High tier is 0.001% of the cohort; the Medium tier is 2.56%; the Low tier is 97.44%. Of the 79,661 Low-tier wallets, 72,135 have a score of zero and 7,526 have a score between 1 and 24. The zero spike is expected: most wallets in the sender/receiver universe have no rule hit at all.

Table 5.4: Final tier distribution and score concentration.

Tier	Wallets	Share	Comment
High (60–100)	1	0.001%	one wallet at score 67
Medium (25–59)	2,094	2.56%	active triage queue
Low (0–24)	79,661	97.44%	72,135 at zero; 7,526 at 1–24
Total	81,756	100%	9,627 with an investigative signal

The top-driver distribution is highly concentrated. R4 is the top driver for 7,443 wallets, R3 for 1,985, R1 for 161, and R2 for 32. The remaining 72,135 wallets have no driver because their score is zero. The dominance of R4 is a consequence of the continuous tail: a large number of wallets receive a small PageRank contribution, and for many of them that small contribution is the only positive term. R4 cannot by itself raise a wallet above 15 points, so these wallets remain Low. R3, with a maximum of 30 points, is the only single rule that

can place a wallet in the Medium tier. This explains why most Medium-tier wallets have R3 as their driver.

Table 5.5 lists the five highest-scoring wallets. The top wallet, `0x9a86deb3b58b18dc192b5ad31066dc6fe3e7c83d`, has a raw score of 66.792896 and a final score of 67. It combines R1 (25), R3 (30), and R4 (11.792896). It is not present in the label store, so it is an unlabelled candidate surfaced by the pipeline. The second wallet, `0xc57f1148855e67763a694f7f2c0e68230adc686e`, has a raw score of 58.963379 and a final score of 59. It combines R1 (25), R2 (20), and R4 (13.963379), and it carries darklist and fraud labels in the training split. The third wallet is a clean R1 + R3 combination with no R4 contribution, scoring exactly 55.

Table 5.5: Five highest-scoring wallets and their rule contributions.

Rank	Address	Raw	Score	Tier	Driver	Contributions (R1/R2/R3/R4/R5)
1	`0x9a86deb3b58b18dc192b5ad31066dc6fe3e7c83d`	66.793	67	High	R3	25 / 0 / 30 / 11.793 / 0
2	`0xc57f1148855e67763a694f7f2c0e68230adc686e`	58.963	59	Medium	R1	25 / 20 / 0 / 13.963 / 0
3	`0x2e449c2c46e0eeab446c59955fa810b7aee1ef0f`	55.000	55	Medium	R3	25 / 0 / 30 / 0 / 0
4	`0x0661474e70b2588029b68c077e8c7adfbcf830db`	44.697	45	Medium	R3	0 / 0 / 30 / 14.697 / 0
5	`0x84d7e3d67ae52a3e332eb7315759da7c25374f33`	44.882	45	Medium	R3	0 / 0 / 30 / 14.883 / 0

The co-occurrence pattern is even more concentrated than the driver distribution. Table 5.6 shows that 9,279 scored wallets hit exactly one rule, 337 hit two, and only 5 hit three. No wallet hit four or five rules. The sole High-tier wallet is one of the five three-rule wallets. This is the arithmetic consequence of the locked weights: one rule can reach Medium only if it is R3 (30) or R1 (25); two rules can reach at most 55 if they are R3 + R1; three rules are needed to cross 60 in most combinations.

Table 5.6: Number of scoring rules hit per wallet.

Scoring rules hit	All wallets	Medium tier	Comment
0	72,135	0	no scored evidence
1	9,279	1,782	single-rule queue
2	337	308	compound evidence
3	5	4	one High and four Medium
4	0	0	not observed
5	0	0	not observed

The single-rule Medium wallets are almost entirely R3 cases. A wallet with only R1 scores 25 and is Medium, but R1-only wallets are rare because a cycle member often also has a sanctions or centrality contribution. A wallet with only R4 scores at most 15 and is Low. This is why the Medium queue is best understood as a sanctions-exposure queue with a smaller number of compound cases, not as a balanced multi-signal queue.

5.7 DASHBOARD RESULTS

The dashboard is the main interface through which the results are inspected. Figure 5.1 shows the priority triage queue, ordered by score. The sole High-tier wallet is at the top, followed by Medium-tier candidates. The queue shows the score, the raw score in parentheses, the driver, the number of signals, the known/split status, and the label source. The second wallet is visibly labelled “darklist, fraud”, which gives the analyst a quick way to distinguish a labelled positive from an unlabelled candidate.

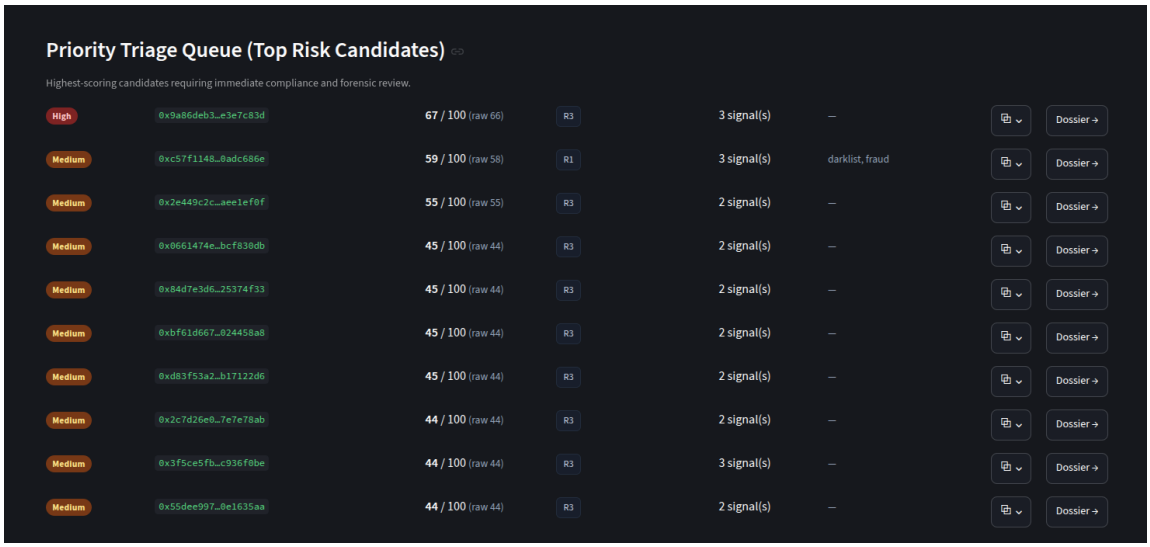


Figure 5.1: Priority triage queue ordered by composite score. The sole High-tier wallet (67) is followed by Medium-tier candidates; the second wallet carries darklist and fraud labels.

Figure 5.2 shows the overview metrics and distributions. The score histogram has a large spike at zero and a thin tail above 25. The signal-frequency chart shows the relative scale of the rules: R4 (7,758) and R3 (2,014) dominate, R1 (163) and R2 (33) are much smaller, and R5 is zero. DBSCAN cluster and Louvain community counts are shown for context; they are not risk signals. The headline "Test Avg Precision" is 61.8%, which is the average precision reported in the next section.

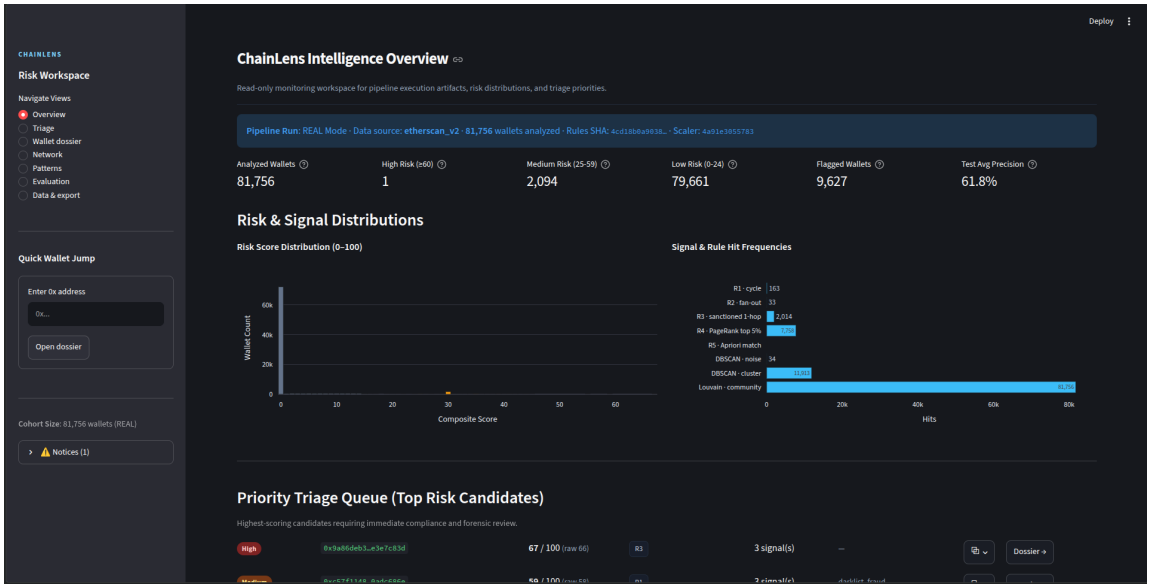


Figure 5.2: Overview metrics with the risk-score histogram and signal-frequency chart. The zero spike reflects wallets with no rule hit; R4 and R3 dominate the signal counts.

Figure 5.3 shows the Triage view with the High and Medium tiers selected. The filter

returns 2,095 matching wallets, which is the one High-tier wallet plus the 2,094 Medium-tier wallets. The table is paginated and can be exported as CSV. The view is the operational counterpart of the score distribution: it lets an analyst move from a population-level count to a specific wallet and then to the dossier.

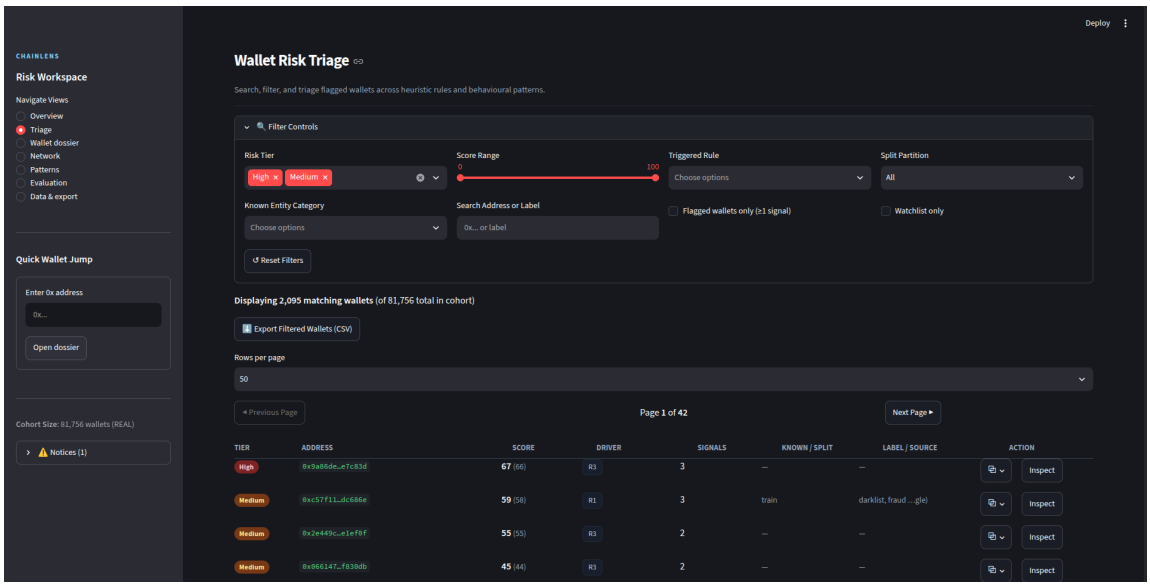


Figure 5.3: Triage view with tier, score, rule, split, category, text, flagged-only, and watchlist filters. The selected High and Medium tiers return 2,095 wallets.

Figure 5.4 shows the dossier for the top wallet. The five rule cards show the weight, a short description, and the trigger status. R1 is triggered for 25 points, R3 for 30 points, and R4 for 11.8 points; R2 and R5 are not triggered. The contribution breakdown repeats the same values as horizontal bars. This is the clearest demonstration of the explainability claim: the final score of 67 is the sum of three visible contributions, not a black-box output.

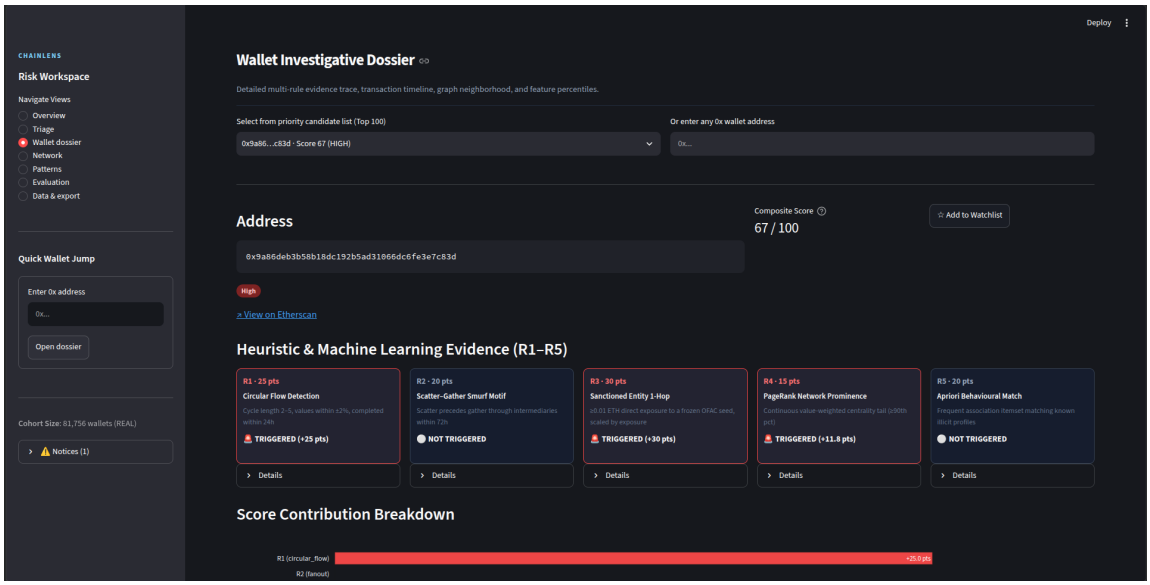


Figure 5.4: Dossier for the top wallet (score 67). The R1, R3, and R4 cards show 25, 30, and 11.8 points respectively; the contribution breakdown matches the trace.

Figure 5.5 shows the transaction evidence sub-tab for the same wallet. The wallet has two cleaned transactions in the fetched window, both with the same counterparty: an inbound transfer of 0.1087 ETH and an outbound transfer of 0.1081 ETH, separated by about 12 minutes on 13 November 2018. The near-equal values in opposite directions are exactly the structure that R1's length-two cycle test is designed to catch. The transaction table provides the hashes and timestamps that an analyst would need to verify the evidence on-chain.

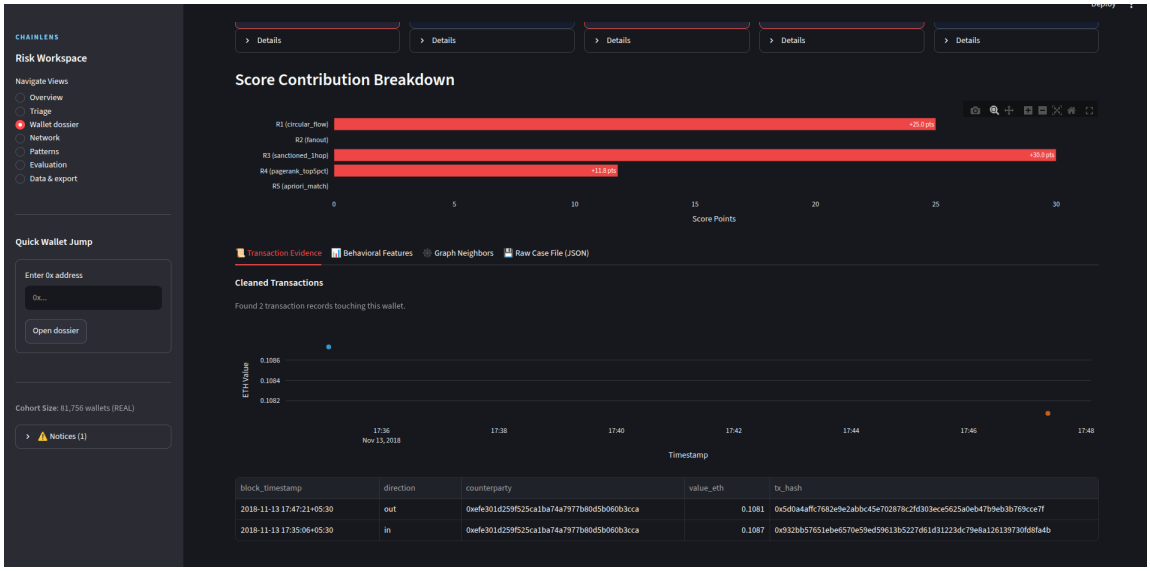


Figure 5.5: Transaction evidence for the top wallet: an inbound 0.1087 ETH transfer and an outbound 0.1081 ETH transfer with the same counterparty, about 12 minutes apart. The near-equal opposing values form the length-two cycle evidence.

Figure 5.6 shows the Patterns view for R1. The table lists 163 flagged wallets with the cycle evidence, the wallet score, and the tier. The scores in the visible rows range from 25 to 40, which is consistent with R1 contributing 25 points and other rules adding smaller amounts. The raw evidence string is verbose, but it is the actual object that the scoring stage consumed; the dashboard does not paraphrase it.

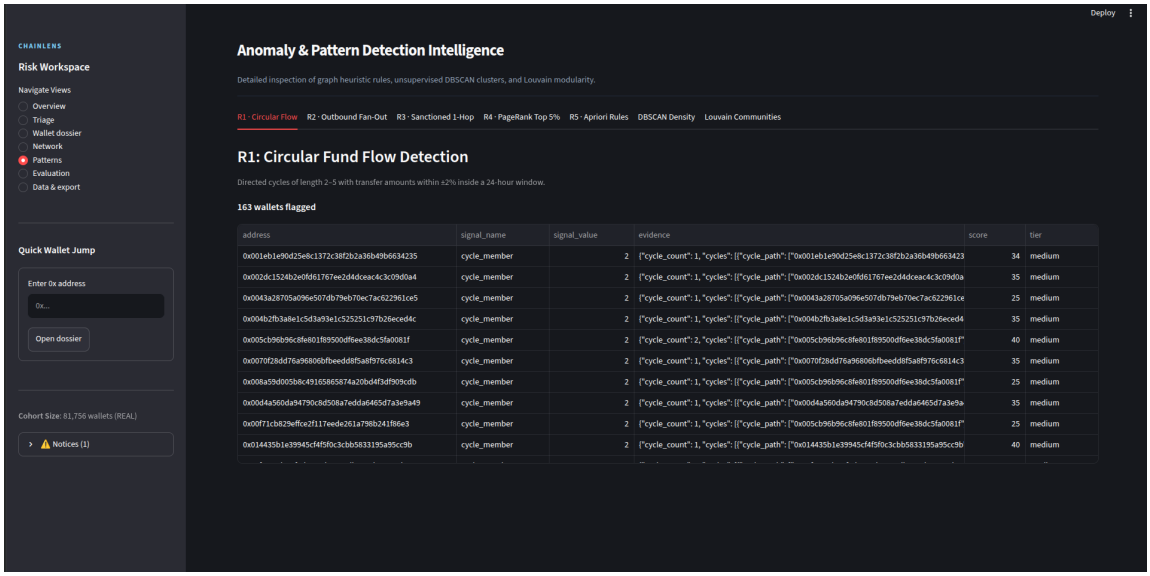


Figure 5.6: Patterns view for R1. The table exposes the 163 flagged wallets with cycle evidence, score, and tier; the evidence strings are the objects consumed by the scoring stage.

Figure 5.7 shows the Network view. The focal wallet is displayed with a two-hop ego-subgraph, capped at 35 nodes; the rendered subgraph has 35 nodes and 39 directed edges. Nodes are coloured by risk tier, and the focal node is highlighted. The view is intended for structural inspection: an analyst can see whether a flagged wallet is a leaf, a hub, or part of a denser component. It is not a scoring input; it is a way to read the graph that produced the score.

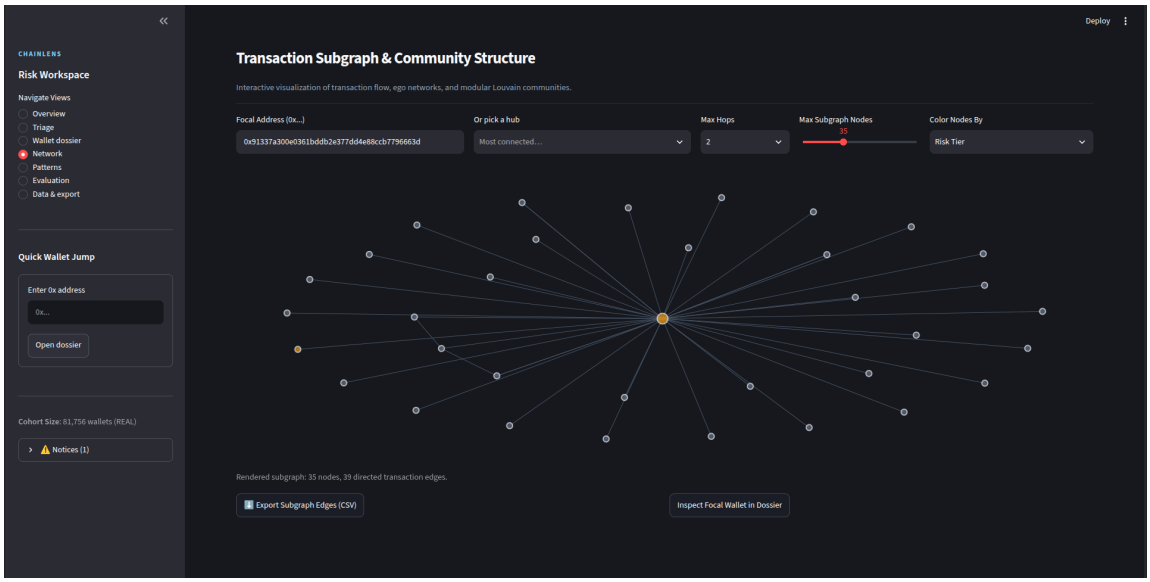


Figure 5.7: Network view with a two-hop ego-subgraph around a focal wallet (35 nodes, 39 directed edges), coloured by risk tier. The view supports structural inspection but does not change the score.

5.8 PERFORMANCE EVALUATION

Table 5.7 reports the two operating points. At the High threshold (score ≥ 60), the test set has no true positives, no false positives, 599 true negatives, and 538 false negatives. Precision, recall, and F1 are all zero. At the Medium threshold (score ≥ 25), the system has 12 true positives, 10 false positives, 589 true negatives, and 526 false negatives. Precision is 54.55%, recall is 2.23%, and F1 is 4.29%. Average precision across all thresholds is 0.6181.

Table 5.7: Operating-point metrics on the held-out case-control test sample.

Metric	**High** ($\tau = 60$)	**Medium** ($\tau = 25$)	**Note**
True positives	0	12	correctly flagged positives
False positives	0	10	flagged negatives
True negatives	599	589	correctly cleared negatives
False negatives	538	526	missed positives
Precision	0.00%	54.55%	$TP/(TP+FP)$
Recall	0.00%	2.23%	$TP/(TP+FN)$
F1	0.00%	4.29%	harmonic mean
Average precision	0.6181		threshold-independent

Table 5.8 presents the same information as confusion matrices. The High-threshold matrix is degenerate in the sense that the predictor never predicts positive; the Medium-threshold matrix has a small positive column. The asymmetry is important: the system is not random, as the average precision of 0.6181 shows, but the fixed High threshold is too strict for the score distribution produced by the locked weights.

Table 5.8: Confusion matrices at the two operating points.

	High ($\tau = 60$)		**Medium** ($\tau = 25$)	
Outcome	**Pred. neg.**	**Pred. pos.**	**Pred. neg.**	**Pred. pos.**
Actual negative	599	0	589	10
Actual positive	538	0	526	12

Figure 5.8 shows the Evaluation view. The two columns report the High and Medium thresholds, with precision, recall, F1, and a confusion-matrix heatmap. The methodology note at the top of the view states that the metrics are sample metrics on a case-control sample, not population estimates. This note is not decorative; it is the correct interpretation of the numbers.

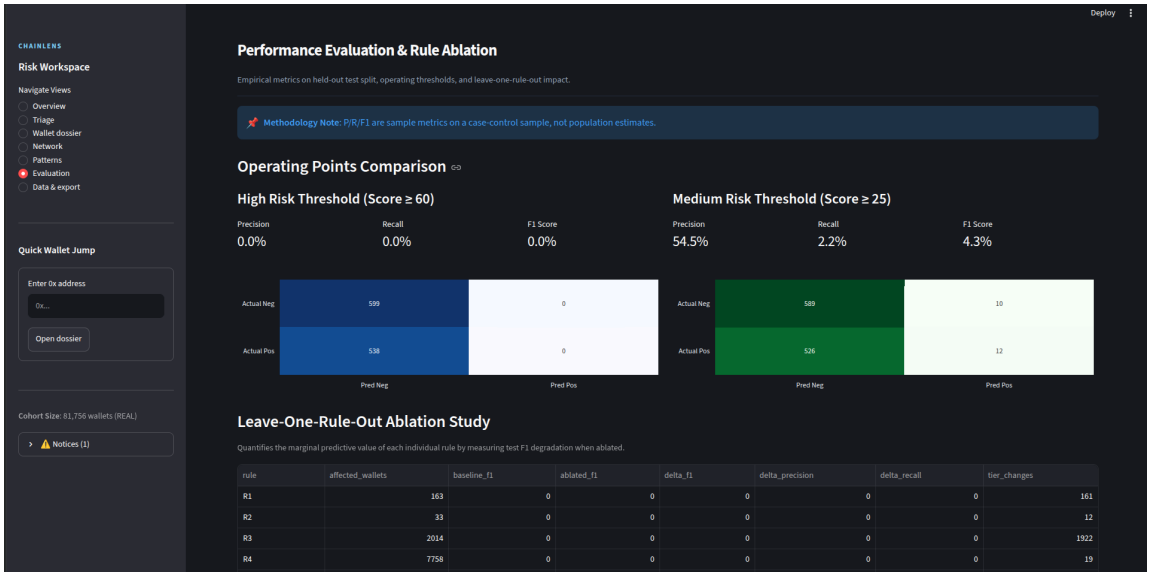


Figure 5.8: Evaluation view showing the High (0/0/0) and Medium (54.5/2.2/4.3) operating points with confusion matrices and the case-control caveat.

The average precision of 0.6181 is the most informative single number in the evaluation. It says that, when wallets are ranked by score, a positive is more likely to appear before a

negative than chance would predict. The High-threshold F1 of zero does not contradict this; it says that the top of the ranking does not reach 60 on this test sample. The Medium threshold recovers a small number of true positives at a precision above one half, but the recall is only 2.23%, so the queue misses almost all positives. A compliance team would have to decide whether a high-precision, low-recall queue is preferable to a lower-precision, higher-recall queue. ChainLens exposes both operating points so that the trade-off is visible.

5.9 ABLATION STUDY

Table 5.9 reports the leave-one-rule-out ablation. Because the baseline High-threshold F1 is zero, every ablated F1 is also zero, and every $\Delta F1$ is zero. The Δ precision and Δ recall columns are likewise zero. Taken alone, this table would suggest that no rule matters. That interpretation is wrong. The ablation is degenerate at the High threshold because the predictor never predicts positive at 60, so removing a rule cannot change a metric that is already zero.

Table 5.9: Leave-one-rule-out ablation at the High threshold.

Rule	Affected	Base F1	Abl. F1	ΔF1	ΔP	ΔR	Tier changes
R1	163	0.000	0.000	0.000	0.000	0.000	161
R2	33	0.000	0.000	0.000	0.000	0.000	12
R3	2,014	0.000	0.000	0.000	0.000	0.000	1,922
R4	7,758	0.000	0.000	0.000	0.000	0.000	19
R5	0	0.000	0.000	0.000	0.000	0.000	0

The tier-change column is the meaningful secondary signal. Removing R3 changes the tier of 1,922 wallets, far more than any other rule. Removing R1 changes 161, R2 changes 12, and R4 changes 19. Removing R5 changes none, because R5 never fired. These counts show that R3 is the load-bearing rule in the current score distribution: it is the only rule that can move a wallet into Medium on its own, and it contributes 30 points to compound cases. R1 is the second most influential rule because its 25 points can also reach the Medium threshold. R2 and R4 are supporting signals at the current thresholds; their contributions are real, but they rarely change a tier without R3 or R1.

Figure 5.9 shows the ablation view. The table is reproduced with the tier-change counts, and the $\Delta F1$ bar chart is flat at zero. The flat chart is an honest visual: it shows that the chosen metric cannot discriminate between rules at this operating point. The tier-change column, which is visible in the same view, is the better guide to rule influence.

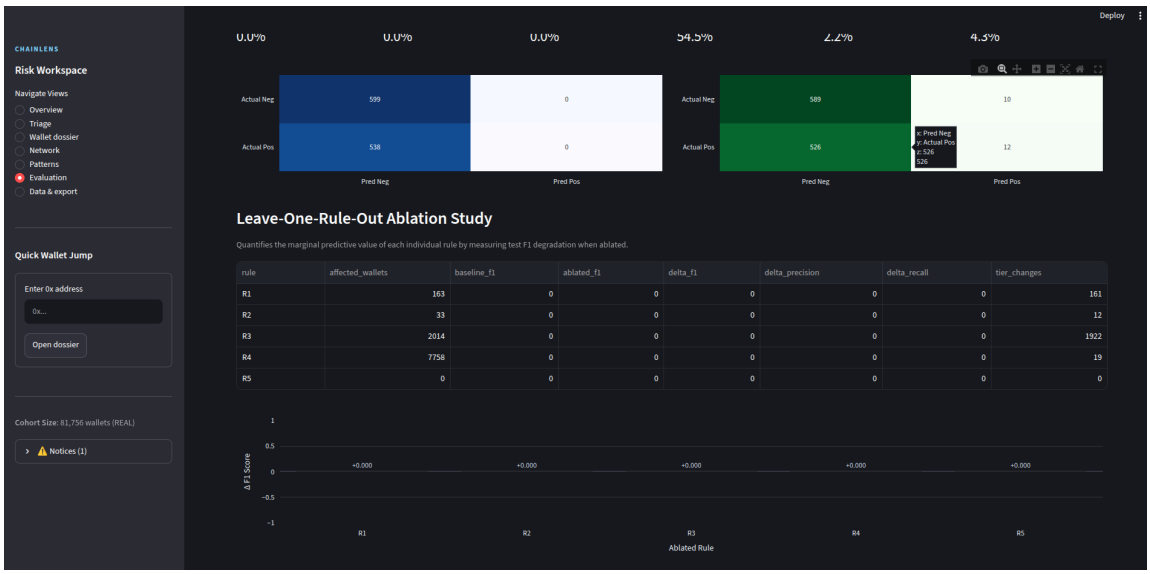


Figure 5.9: Ablation view. All $\Delta F1$ values are zero because the baseline High-threshold F1 is zero; the tier-change column shows that R3 is the dominant rule (1,922 changes).

5.10 DISCUSSION

The results support four conclusions. First, the pipeline is operational end to end. It ingests real Ethereum data, builds a graph, detects patterns, scores wallets, evaluates the scores, and exposes the evidence in a dashboard. The artifact set is complete enough to reproduce the reported numbers without re-fetching data. Second, the score distribution is extremely skewed. Most wallets have no rule hit, a small number have one, and only five have three. The High tier contains a single wallet. This is not an accident of implementation; it is the arithmetic consequence of the locked weights and the conservative gates.

Third, R3 dominates the operational queue. It flags 2,014 wallets, drives 1,985 scores, and changes 1,922 tiers when ablated. This is consistent with the design decision to give direct sanctions exposure the highest weight. It also means that the queue is sensitive to the sanctions snapshot and to the minimum-value gate. A different snapshot, a different dust threshold, or a different exposure floor would change the queue materially. The trace makes that sensitivity inspectable, but it does not remove it.

Fourth, R5 is inactive and R4 is a weak supporting signal. R5's best candidate lift is 1.394, below the 2.0 threshold. R4 flags 7,758 wallets but only changes 19 tiers, because its maximum contribution is 15 and most of its tail contributions are small. These are not failures; they are findings about the data and the contract. A five-rule design can operate as a four-rule system, and a centrality rule can be useful for ranking without being decisive for tiering.

The evaluation has three important limitations. The test set is a case-control sample with a 47.3% positive rate, so precision is not a population estimate. The High threshold produces no true positives on this sample, which makes the High-threshold F1 and the F1-based ablation degenerate. The labels are drawn from sources with different snapshot dates, and some negatives may have become positive after the label date. These limitations do not invalidate the pipeline, but they bound what the numbers can be used to claim.

The dashboard evidence is consistent with the trace. The top wallet's dossier shows 25 + 30 + 11.8 = 66.8 raw points, rounded to 67. The transaction tab shows the two opposing transfers that form the length-two cycle. The R1 pattern table shows 163 wallets with cycle evidence. The evaluation view shows the same confusion matrices as the JSON. This consistency is the practical test of explainability: the interface does not tell a different story from the artifacts.

5.11 PROVENANCE AND EXPORT

Figure 5.10 shows the Data & Export view. The artifact inventory lists the scores, traces, flags, graph edges, features, labels, splits, known addresses, clean transactions, evaluation, and provenance files, with sizes, modification times, and presence status. The provenance panel shows the run identifier, mode, seed, source, chain, and window. The export bundle allows an analyst to download the complete artifact archive.

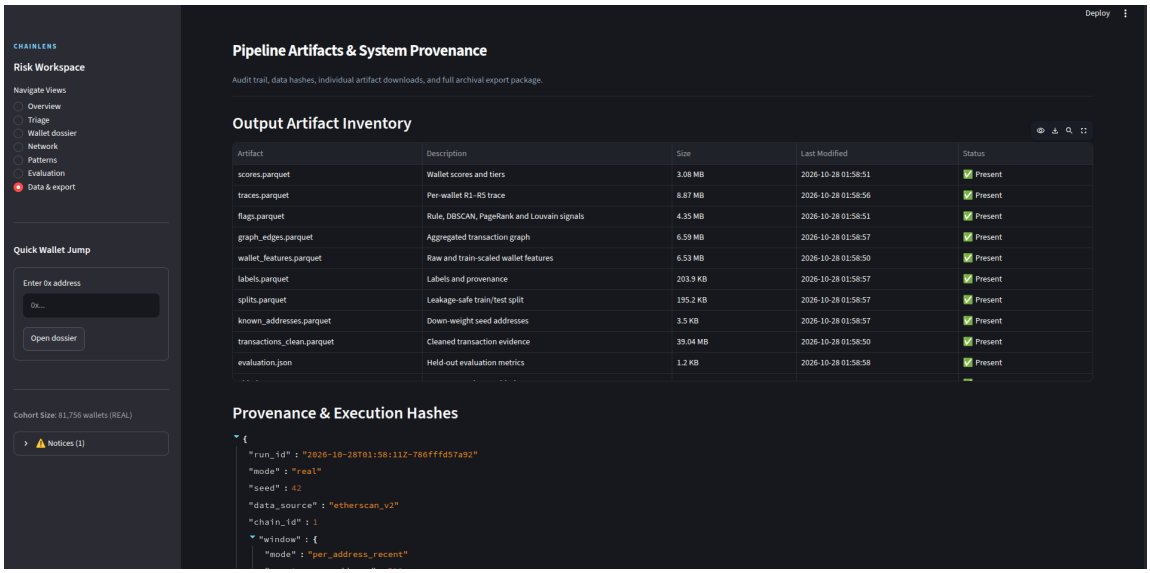


Figure 5.10: Data & Export view with the artifact inventory and provenance JSON. The view is the audit entry point for reproducing the reported run.

The inventory is not a substitute for a reproducibility package. A complete package would include the cached transactions, the exact configuration files, the Python environment lock, and a script that regenerates the artifacts. The repository contains most of these elements, and the provenance file records the hashes that bind the run to its configuration. The remaining gap is the size of the cached transaction data and the API key required to refresh it. This is a practical constraint, not a conceptual one.

5.12 SUMMARY

The implementation chapter reports a complete real-mode run: 81,756 wallets, 99,706 edges, 163 R1 members, 33 R2 members, 2,014 R3 members, 7,758 R4 members, and no R5 rules. The score distribution is one High, 2,094 Medium, and 79,661 Low. The High-threshold test metrics are zero; the Medium-threshold metrics are 54.55% precision, 2.23% recall, and 4.29% F1; average precision is 0.6181. The ablation is degenerate at the High threshold, but tier changes identify R3 as the dominant rule. The dashboard exposes the evidence behind these numbers in seven read-only views. The next chapter concludes with the limitations, future scope, and the project's SDG 16 contribution.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

This report presented ChainLens, an explainable Ethereum wallet risk-scoring pipeline, and evaluated it on an artifact-backed real-mode run of 28 October 2026. The system ingested 379,023 raw records, retained 368,113 clean transactions, engineered 14 behavioural features for 81,756 wallets, and built a directed graph with 99,706 edges. Five grounded rules produced a transparent score: circular flow, scatter–gather smurfing, one-hop sanctions exposure, continuous value-weighted PageRank, and Apriori profile matching. Every score was accompanied by a trace containing the rule hits, evidence strengths, point contributions, and downweight details.

The detection results were uneven, and the unevenness is informative. R1 found 97 length-two cycles involving 163 wallets. R2 confirmed 33 scatter–gather wallets after rejecting scatter-only candidates. R3 flagged 2,014 wallets with a mean evidence strength of 0.963. R4 placed 7,758 wallets in the continuous PageRank tail. R5 produced no rules because the best candidate lift of 1.394 was below the locked threshold of 2.0. DBSCAN produced three clusters and 34 noise points; Louvain produced 18,512 communities. The final distribution was one High-tier wallet, 2,094 Medium-tier wallets, and 79,661 Low-tier wallets.

The evaluation was deliberately reported without embellishment. At the High threshold, precision, recall, and F1 were all zero on the 538-positive, 599-negative case-control test sample. At the Medium threshold, precision was 54.55%, recall was 2.23%, and F1 was 4.29%. Average precision was 0.6181. The High-threshold ablation was degenerate because the baseline F1 was zero, but the tier-change counts identified R3 as the load-bearing rule (1,922 changes), followed by R1 (161), R2 (12), and R4 (19). These results do not show that ChainLens is a production-ready AML system. They do show that it is a reproducible, inspectable triage system whose failure modes are visible.

The explainability claim is supported by the dashboard and the traces. The top wallet's score of 67 is the sum of 25 points for R1, 30 points for R3, and 11.79 points for R4. The

dossier shows the same three contributions, and the transaction view shows the two opposing transfers that form the length-two cycle. The evaluation view shows the same confusion matrices as the JSON. An analyst can move from a population count to a wallet to a transfer without leaving the artifact set.

6.2 CONTRIBUTIONS

The project makes four contributions. First, it provides a complete, reproducible pipeline for Ethereum wallet risk scoring that runs from ingestion to dashboard on a fixed artifact set. Second, it implements a paper-grounded rule vocabulary with concrete evidence objects, including the gather phase of the smurf motif, dusting-resistant sanctions exposure, and a continuous PageRank tail with hub exclusion. Third, it demonstrates a trace-first scoring design in which the decision record, not the model, is the primary object; the dashboard is a read-only view of that record. Fourth, it reports a negative result honestly: a five-rule contract can operate as a four-rule system, a continuous centrality signal can be a weak tier driver, and a high threshold can produce zero true positives on a case-control sample.

The code, configuration, and dashboard also serve as a teaching artifact: small enough for a student to understand, yet complete enough to expose label quality, class imbalance, threshold choice, hub suppression, and the gap between a ranking metric and an operational queue.

6.3 LIMITATIONS

The limitations fall into four groups. The first is data. The transaction set is a cached snapshot, not a live feed. The per-address window of 500 transactions may miss older cycles and long-range relationships. The labels come from sources with different snapshot dates, and the negative pool is a sample rather than a verified clean set. An address labelled negative in 2021 may have become positive later.

The second is coverage. R1 can only see cycles inside the fetched cohort. R2 uses absolute thresholds rather than relative velocity. R3 is strictly one hop and does not implement two-hop decay. R4 is a centrality measure, not a risk measure, and its known-entity exclusion depends on a registry that may be incomplete. R5 is inactive on this run. DBSCAN and Louvain are

descriptive signals, not evidence of wrongdoing.

The third is evaluation. The test set is a case-control sample with a 47.3% positive rate, so the reported precision is not a population estimate. The High threshold produces no true positives, which makes the High-threshold F1 and the F1-based ablation degenerate. The ablation is a re-scoring exercise, not a retraining exercise, because the scoring contract is not learned. Tier changes are a useful secondary signal, but they are not a substitute for a proper cost-sensitive evaluation.

The fourth is operational. The system is batch-oriented and does not provide alerting, case management, user authentication, or a feedback loop from analyst decisions. It does not establish legal liability, and a High-tier score is a prioritisation signal rather than proof of wrongdoing. The dashboard is read-only by design, but a production deployment would need access control, audit logging, and a process for correcting labels.

6.4 FUTURE SCOPE

Several extensions follow directly from the limitations. The most valuable is an on-demand wallet analyser: given one address, fetch its ego-subgraph, evaluate the same rules, and produce a dossier without running the full cohort. The existing modules already expose most of the required logic; the missing pieces are neighbour expansion, a frozen scaler, frozen PageRank reference percentiles, and a serialised R5 rule set.

A second extension is a stronger sanctions model. Two-hop exposure with decay, cross-chain bridge awareness, and a time-aware sanctions snapshot would improve coverage without abandoning the one-hop core. The exposure fraction could be calibrated against known cases rather than set with a fixed 0.10 floor. A third extension is a better R5. Mining on a larger positive set, using a constrained item vocabulary, and validating rules on a temporal holdout would give the association rule a fair chance to fire. The current zero result should not be interpreted as evidence that association mining is useless for wallet profiles; it is evidence that this dataset and this threshold did not support it.

A fourth extension is evaluation. A temporal split, a prevalence-adjusted precision estimate, and a cost-sensitive analysis of the Medium threshold would make the operational trade-off clearer. The current case-control sample is adequate for comparing rules, but not for estimating the precision a bank would see. A fifth extension is a temporal graph that

preserves the order of transfers at the edge level, which would let R1 and R2 reason about paths rather than aggregated edges.

A sixth extension is human-in-the-loop learning: analyst decisions could recalibrate the known-entity registry or the thresholds, but any learned adjustment should be recorded as an inspectable rule contribution rather than an opaque rescoring.

6.5 SUSTAINABLE DEVELOPMENT GOAL 16

The project is mapped primarily to Sustainable Development Goal 16: peace, justice, and strong institutions. Target 16.4 calls for a significant reduction in illicit financial and arms flows; target 16.5 calls for substantially reducing corruption and bribery; target 16.6 calls for effective, accountable, and transparent institutions; and target 16.10 calls for public access to information and fundamental freedoms. ChainLens contributes to these targets in a modest but concrete way. It makes sanctions exposure, circular flows, and smurf-like dispersion visible at the wallet level, and it attaches each signal to evidence that an analyst can inspect. It does not replace a compliance programme, but it strengthens the evidentiary basis on which a compliance decision can be made.

The transparency contribution is as important as the detection contribution: an unexplainable score is difficult to challenge, and an unchallengeable score is a poor fit for accountable institutions. ChainLens records the rule contract, the strength of each signal, the contribution of each rule, and the downweight decision. It reports a degenerate High-threshold result and a case-control caveat instead of presenting a favourable number without context. This is a small example of what accountable algorithmic governance looks like in practice.

The secondary mappings are Goals 9 and 4. Goal 9 concerns resilient infrastructure and innovation; a transaction-graph pipeline that can be audited is a contribution to trustworthy digital financial infrastructure. Goal 4 concerns quality education; the open, reproducible implementation and the evidence-oriented dashboard make blockchain forensics and graph mining easier to teach. The primary mapping remains Goal 16 because the project's purpose is not merely to build a better classifier, but to make the classification defensible.

6.6 CONCLUDING REMARKS

ChainLens shows that explainability is an architectural choice. By keeping the rule vocabulary small, locking the weights, recording the evidence, and separating computation from presentation, the system produces a risk score that can be reconstructed and challenged. The cost of that choice is visible: the High tier is nearly empty, recall is low, and one of the five rules is inactive. Those are real costs, and the report states them. The benefit is a triage system that an analyst can use without surrendering judgement to an opaque model. In the context of SDG 16, that trade-off is the right place to start.

BIBLIOGRAPHY

[1] United Nations. Transforming our world: The 2030 agenda for sustainable development, 2015. A/RES/70/1.

[2] Financial Action Task Force. Updated guidance for a risk-based approach to virtual assets and virtual asset service providers. `https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Guidance-rba-virtual-assets-2021.html`, 2021. Accessed October 2026.

[3] U.S. Department of the Treasury, Office of Foreign Assets Control. Specially designated nationals and blocked persons list: Digital currency addresses. `https://sanctionssearch.ofac.treas.gov/`, 2026. Accessed October 2026.

[4] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD 2019 Workshop on Anomaly Detection in Finance*, 2019. arXiv:1908.02591.

[5] D. Lin et al. Modeling and understanding Ethereum transaction records via a complex network approach. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2020.

[6] J. Lorenz et al. Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF '20)*. ACM, 2020.

[7] Etherscan. Etherscan API V2 documentation: Account and transaction endpoints. `https://docs.etherscan.io/`, 2026. Accessed October 2026.

[8] NetworkX Developers. NetworkX documentation: PageRank, strongly connected components, and cycle detection. `https://networkx.org/documentation/stable/`, 2026. Accessed October 2026.

[9] DuckDB Foundation. DuckDB documentation: SQL, Parquet, and decimal types. `https://duckdb.org/docs/`, 2026. Accessed October 2026.

[10] Fabian Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[11] Sebastian Raschka. MLxtend: Providing machine learning and data science utilities and extensions to python's scientific computing stack. *Journal of Open Source Software*, 3(24):638, 2018.

[12] Friedhelm Victor and Andrea M. Weintraud. Detecting and quantifying wash trading on decentralized cryptocurrency exchanges. In *Proceedings of the Web Conference 2021 (WWW '21)*. ACM, 2021.

[13] Y. Hu et al. Characterizing and detecting money laundering activities on the bitcoin network, 2019. arXiv:1912.12060.

[14] Michele Starnini et al. Smurf-based anti-money laundering in time-evolving transaction networks. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2021)*. Springer, 2021.

[15] E. Liu et al. Evasion under blockchain sanctions, 2025. arXiv:2507.11721.

[16] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, 1999.

[17] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pages 487–499, 1994.

[18] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231, 1996.

[19] M. Ostapowicz and K. Żbikowski. Detecting fraudulent accounts on blockchain: A supervised approach. In *Web Information Systems Engineering – WISE 2019*. Springer, 2019.

[20] J. Hirshman, Y. Huang, and S. Macke. Unsupervised approaches to detecting anomalous behavior in the bitcoin transaction network. Cs229 technical report, Stanford University, 2013.

[21] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.

[22] S. Phetsouvanh, F. Oggier, and A. Datta. Egret: Extortion graph exploration techniques in the bitcoin network. In *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2018.

[23] Streamlit. Streamlit documentation: Multipage apps and dataframes. `https://docs.streamlit.io/`, 2026. Accessed October 2026.